

Universidad Nacional de Colombia
Sede Manizales



Informe técnico

Desarrollo de módulos de inteligencia artificial informados por la física y apoyados por agentes para el estudio de criticidad e intervenciones en redes de nivel de tensión 2 de
CHEC

Contrato: CRW254513 de 2024

Acta: Acta de Trabajo No. 1

Entidad: Central Hidroeléctrica de Caldas S.A. E.S.P. – CHEC

Contratista: Universidad Nacional de Colombia Sede Manizales

Fecha: 25 de agosto de 2026

Índice

1	Resumen ejecutivo	3
2	Identificación del proyecto	4
3	Referencia contractual	4
4	Actividades realizadas	5
4.1	Cambios de alcance respecto del informe anterior	6
5	Arquitectura de la solución	6
5.1	Componentes en operación	6
5.2	Cadena de ejecución y separación de responsabilidades	7
5.3	Estructura del repositorio	7
6	Tableros de visualización y ampliación de datos climáticos	8
6.1	Nube por vano y clima	9
6.1.1	Captura de datos climáticos con /clima	10
6.2	Agrupamiento de vanos	11
6.3	Trayectorias de circuitos	12
6.4	Trayectorias de vanos	13
6.5	Del vano al grupo de circuito	14
7	Modelo de inteligencia artificial predictiva M-GCECDL	16
7.1	Unidad de análisis: la bolsa vano $\times$ ventana	16
7.2	Arquitectura del modelo	17
7.3	Variables de entrada	19
7.4	Formulación matemática	20
7.5	Grafo de restricciones físicas	22
7.5.1	Alcance de cada modo dentro del grafo	22
7.6	Aplicación sobre datos nuevos	24
7.7	Protocolo de validación y resultado	25
7.8	Cómo se comporta el modelo: curvas, matriz de confusión y un caso completo	27
7.8.1	Curvas de entrenamiento: ¿está aprendiendo?	27
7.8.2	Curvas ROC: ¿separa bien los vanos críticos de los que no lo son?	28
7.8.3	Matriz de confusión: ¿en qué se equivoca cuando se equivoca?	29
7.8.4	Un caso completo: de la inferencia al diagnóstico y a la simulación	30
7.9	Dónde se entrena	32
8	Simulador contrafactual “¿Qué pasa si...?”	34
8.1	Flujo de una simulación	35
8.2	Clases de variable y criterios de separación	37
8.3	Incorporación de los costos por actividad de contrato	38
8.4	Archivo y recuperación de una simulación	39
8.5	Rendimiento medido	40
9	Agentes e informes automáticos	40
9.1	Qué es una habilidad y cómo se ejecuta	41
9.2	Informe por circuito	41
9.3	Informe gerencial por banda de riesgo	42

10	Migración a Azure Databricks	43
10.1	La interfaz de línea de comandos como estrategia central	43
10.2	Secuencia de ejecución	43
10.3	Trazabilidad del despliegue	44
11	Instalación y puesta en marcha	44
11.1	Los seis pasos	45
11.2	Requisitos de máquina, medidos	45
11.3	Las cuatro listas de dependencias	46
11.4	Comandos de instalación y mantenimiento	47
11.5	Portabilidad entre entornos de agente	47
11.6	Primer recorrido de uso	48
12	Grafo de conocimiento del proyecto	49
13	Pruebas y validación	50
14	Estimación de costo operativo en Azure Databricks	50
15	Entregables y enlaces	52
16	Conclusiones	52

1. Resumen ejecutivo

El presente informe describe el estado vigente de los desarrollos asociados al Acta de Trabajo No. 1 del Contrato CRW254513 de 2024, orientados a construir módulos de inteligencia artificial informados por la física y apoyados por agentes para el estudio de la criticidad y la efectividad de las intervenciones en redes de nivel de tensión 2 de CHEC.

La solución entregada se compone de cinco aplicaciones locales, un modelo predictivo entrenado, un simulador contrafactual y un conjunto de agentes para la generación de informes técnicos. Todos los componentes se ejecutan de forma local sobre 127.0.0.1, sin exponer servicios a la red, y la totalidad de la cadena puede desplegarse en Azure Databricks mediante un único comando.

Cuatro decisiones de diseño gobiernan el conjunto y explican su comportamiento:

- **La unidad de análisis es la bolsa vano $\times$ ventana.** El modelo M-GCECDL no puntúa un evento aislado ni un circuito completo, sino la celda formada por un vano y una ventana temporal de treinta días. Esa ventana es un *período de análisis*: el intervalo sobre el cual se acumula el UITI del vano y sobre el cual el simulador evalúa una intervención. No corresponde al plazo administrativo de emisión de una orden de trabajo. Lo que la cadena entrega a la operación es el vano priorizado; la ventana indica en qué período se observó ese comportamiento.
- **La clase de criticidad no procede de la red neuronal.** Las cuatro etiquetas — Bajo, Medio, Medio-Alto y Alto — se generan por agrupamiento (*clustering* con *K-medias*) sobre un plano de dos ejes: el número de eventos observados del vano frente a su UITI acumulado. Ese mapa de grupos se calcula una sola vez, queda guardado dentro del archivo del modelo y no se vuelve a recalcular. Cada vez que el sistema arranca compara el mapa guardado con el que está a punto de usar y, si no coinciden, se detiene con un error en lugar de continuar en silencio. Gracias a ese control, el mapa con el que se mide lo observado y el mapa con el que se mide lo simulado son siempre el mismo, y ambos resultados quedan comparables por construcción.
- **Las relaciones entre variables se declaran, no se aprenden.** Un grafo de restricciones físicas de 156 relaciones dirigidas, definido en código y almacenado dentro del artefacto entrenado, delimita qué asociaciones puede utilizar el modelo.
- **El cálculo es determinista y el razonamiento es del agente.** No existe ninguna llamada a servicios externos de modelos de lenguaje desde el código Python del proyecto: Python prepara, calcula, valida y renderiza; el agente interpreta, sujeto a contratos de salida verificables.

El simulador contrafactual responde, para un vano y una ventana concretos, qué variable modificar y hasta qué valor para que el vano descienda de grupo de criticidad, presenta el costo de la intervención con el catálogo de actividades del contrato de CHEC y permite archivar cada ejecución en un informe HTML autocontenido acompañado de un registro de pocos kilobytes que la reconstruye por completo. Los agentes documentan esos mismos resultados en informes por circuito y en síntesis por banda de riesgo.

2. Identificación del proyecto

Tabla 1: Datos generales del proyecto

Campo	Información
Proyecto	Simulador de criticidad CHEC para análisis de fallas, impacto por vano e intervenciones en redes de nivel de tensión 2.
Entidad / cliente	Central Hidroeléctrica de Caldas S.A. E.S.P. – CHEC.
Contratista / equipo	Universidad Nacional de Colombia Sede Manizales, con apoyo del Laboratorio de Inteligencia Artificial.
Contrato	CRW254513 de 2024.
Acta	Acta de Trabajo No. 1.
Periodo del proyecto	Duración de 104 días calendario; inicio del mes 1: 27 de abril de 2026.

3. Referencia contractual

El Contrato CRW254513 de 2024 tiene por objeto generar para la empresa el desarrollo mediante estudios en sus actividades misionales en temas de orden científico, tecnológico e investigativo, innovación y formación en áreas integrales del sector eléctrico.

Tabla 2: Resumen contractual del Acta de Trabajo No. 1

Elemento	Descripción
Contrato	CRW254513 de 2024.
Acta	Acta de Trabajo No. 1.
Objeto del acta	Desarrollar módulos de inteligencia artificial informados por la física y apoyados por agentes para el estudio de la criticidad y la efectividad de las intervenciones en las redes de nivel de tensión 2 de CHEC.
Alcance técnico	Desarrollo de un módulo de simulación contrafactual para analizar escenarios tipo “qué pasa si”, incluyendo reposición de activos críticos y mantenimiento, con estimación proyectada de SAIDI, SAIFI y costos de mantenimiento y reposición, incorporando incertidumbre mediante técnicas probabilísticas y análisis de sensibilidad.
Alcance de infraestructura	Apoyo en contenerización mediante Docker, orquestación en Kubernetes o equivalentes, integración con bases de datos y servicios corporativos, mecanismos de autenticación y control de acceso, y configuración de entornos de desarrollo, pruebas y producción con trazabilidad, respaldo y monitoreo.
Objetivo específico 1	Desarrollar un simulador inteligente de escenarios técnico–económicos con módulo predictivo y chatbot inteligente, simulación contrafactual predictiva con restricciones físicas y evaluación económica dinámica.
Objetivo específico 2	Apoyar la migración y adaptación de las soluciones desarrolladas a la infraestructura de nube corporativa CHEC–EPM, garantizando escalabilidad, seguridad, interoperabilidad y sostenibilidad operativa.
Metodología	Entradas de información; actualización del Prototipo 1; simulación contrafactual con restricciones físicas; evaluación económica; resultados del simulador; comparación y soporte a decisiones de inversión.
Resultado esperado	Simulador inteligente de escenarios técnico–económicos con visualización georreferenciada y tabular, comparación entre estado actual y escenarios intervenidos, y generación de medidas cuantitativas de mejora técnica y ahorro potencial.
Actualización de arquitectura	Migración a infraestructura corporativa CHEC–EPM, contenerización, integración con bases corporativas, actualización normativa automática, versionamiento y mecanismos de seguridad empresarial.

Elemento	Descripción
Entregables	Informe técnico con descripción de los desarrollos; repositorio GitHub con códigos desarrollados y documentados; pruebas piloto sobre cuadernos de Python.
Criterios de aceptación	La entrega y aceptación por parte de CHEC de los informes técnicos habilita los hitos de pago definidos para el acta.
Obligaciones técnicas	Entregar información, procedimientos o cálculos que permitan a CHEC actualizar los resultados en cualquier momento.

Tabla 3: Cronograma de avance del acta, por etapa de la metodología

Etapa	Actividad	Periodo
Ajustes del módulo M-GCECDL y agentes	Actualización de librerías, datos 2025 y flujo local.	Mes 1
Ajustes del módulo M-GCECDL y agentes	Actualización de contexto técnico, normativa y documentación.	Meses 1–2
Simulación contrafactual	Diseño de arquitectura del simulador.	Mes 1
Simulación contrafactual	Modelado probabilístico con restricciones físicas.	Meses 2–3
Simulación contrafactual	Escenarios “qué pasa si”, UITI por vano y clima extremo.	Meses 2–4
Evaluación económica dinámica	Costos, retorno de inversión y clasificación de intervenciones.	Meses 2–4
Visualización y despliegue	Resultados georreferenciados y tabulares e infraestructura en entorno local y nube de CHEC.	Meses 1–4
Validación final	Pruebas técnicas y ajuste fino.	Mes 4

4. Actividades realizadas

Las actividades ejecutadas se agrupan en seis frentes de trabajo, cuyo detalle operativo se desarrolla en las secciones siguientes.

- **Tableros de visualización y ampliación de datos climáticos.** Se consolidaron cuatro visores interactivos — nube por vano y clima, agrupamiento de vanos, trayectorias de circuitos y trayectorias de vanos — gobernados por un menú de escritorio común, y el comando `/clima`, que amplía la base de eventos con variables meteorológicas de Open-Meteo bajo tres validaciones de cuota.
- **Modelo predictivo M-GCECDL sobre bolsas vano $\times$ ventana.** Se construyó y validó un regresor de instancias múltiples que puntúa la celda (circuito, vano, ventana) y deriva su clase de criticidad de una geometría de agrupamiento congelada, con validación cruzada agrupada por vano y un criterio de aceptación prerregistrado.
- **Simulador contrafactual por vano.** Se desarrolló el tablero que compara la criticidad observada contra la simulada sobre el mismo circuito y la misma ventana, propone un diagnóstico de intervención sobre los vanos seleccionados y permite archivar y recuperar cada ejecución.
- **Referencia económica por ítems de contrato.** Se incorporó el libro de precios de actividades de mantenimiento de CHEC — 142 actividades con su costo unitario — como cotización explícita del plan que declara el usuario, tanto en el simulador como en el informe automático por circuito.

- **Agentes e informes automáticos.** Se consolidó el ecosistema de tres roles de agente y tres comandos de informe, con contratos de salida validados, trazabilidad por afirmación y proyección de cada circuito a un repositorio de conocimiento navegable.
- **Despliegue en nube e instalación asistida.** Se unificó la migración a Databricks en un solo comando de ocho etapas conducido por la interfaz de línea de comandos, y se construyó el comando de instalación local que diagnostica la máquina e instala únicamente los componentes faltantes.

4.1. Cambios de alcance respecto del informe anterior

Tres componentes reportados en el informe previo se retiraron del flujo activo. Se registran de forma explícita para evitar que una comparación entre informes los interprete como omisiones.

Tabla 4: Componentes retirados y su reemplazo en el flujo vigente

Componente retirado	Motivo y reemplazo
Clasificador de impacto por evento	La línea que clasificaba el evento aislado se retiró junto con su búsqueda de hiperparámetros. La denominación M-GCECDL designa desde entonces el módulo de instancias múltiples descrito en la Sección 7, que opera sobre la unidad de intervención.
Tablero AI/BI de Lakeview y tablas Delta	La copia del dominio en tablas y vistas dejó de tener consumidores una vez que las aplicaciones Databricks pasaron a leer directamente los archivos del Volume. El despliegue transfiere hoy tres grupos de artefactos, no un modelo de datos completo.
Relevancia de variables por lote	El barrido sobre las 111.233 celdas producía una hoja de cálculo que ningún flujo consumía. La misma pregunta se responde de forma interactiva y acotada a la selección del usuario en el simulador, y de forma documental en el informe por circuito.

Adicionalmente, los cuadernos de análisis se convirtieron en aplicaciones de escritorio: el repositorio conserva un único cuaderno versionado, el que entrena el modelo, y los cuatro visores restantes se ejecutan como HTML precalculado.

5. Arquitectura de la solución

5.1. Componentes en operación

La solución expone cinco aplicaciones, cada una con su propio entorno de Python y su puerto fijo, gobernadas por un menú de control. Los cuatro visores se publican como HTML estático y se cargan en menos de un segundo, porque su contenido se calcula por adelantado y el navegador únicamente filtra. El simulador constituye la excepción: cada simulación requiere una inferencia nueva del modelo, de modo que mantiene un intérprete de Python en ejecución.

Tabla 5: Aplicaciones locales, puerto asignado y modo de ejecución

Aplicación	Puerto	Modo de ejecución
CriticidadCHEC (menú)	8800	Servidor de control; inicia, supervisa y detiene los demás servicios.
Nube por vano y clima	8801	HTML estático.
Agrupamiento de vanos	8802	HTML estático.
Trayectorias de circuitos	8803	HTML estático.
Trayectorias de vanos	8804	HTML estático.
Simulador “¿Qué pasa si...?”	8866	Voilà con núcleo de Python en ejecución.

Cada aplicación calcula una huella criptográfica del árbol de sus insumos y se reconstruye de forma autónoma cuando alguno de ellos cambia. Esa verificación cuesta del orden de 1,4 ms, por lo que se ejecuta en cada arranque sin penalización perceptible.

5.2. Cadena de ejecución y separación de responsabilidades

La Figura 1 presenta la cadena completa. El principio que la gobierna se mantiene sin excepción: **no existe ninguna llamada a servicios externos de modelos de lenguaje desde el código Python del proyecto**. Python realiza únicamente trabajo determinista — lectura de datos, construcción de celdas, inferencia del modelo, cálculo de escenarios, validación de contratos y renderizado —, mientras que el razonamiento en lenguaje natural lo ejecuta el entorno de agente que invoca el flujo.

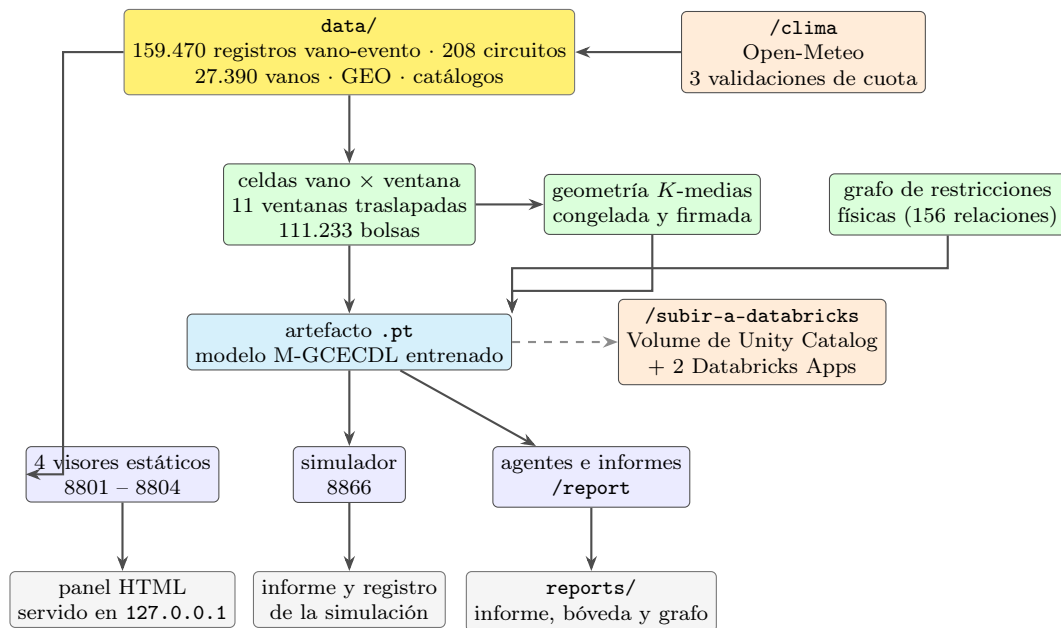


Figura 1: Cadena de ejecución del proyecto. Los insumos de `data/` producen las celdas vano × ventana; sobre ellas se ajusta la geometría de agrupamiento y se entrena el modelo, que queda almacenado en un único artefacto. Ese artefacto alimenta el simulador y los agentes; los cuatro visores estáticos se construyen directamente de los datos. La ruta punteada corresponde al despliegue en Databricks, que replica la misma cadena con sus propios datos.

5.3. Estructura del repositorio

El árbol siguiente omite los archivos generados y las dependencias, y presenta la ubicación del código que se lee y se modifica. Donde aparece un conteo, corresponde al número de archivos

versionados de esa carpeta.

```

chec-local-uiti-vano-interpreter/
|-- src/                                # el codigo que ejecutan todos los flujos
|   |-- chec_local_interpreter/        # motor del analisis y de los informes - 55 archivos
|   |   |-- agent_tools/               # lo que los agentes pueden invocar
|   |   |-- llm/ - prompt_assets/      # contratos y plantillas de los agentes
|   |   |-- clima_engine.py            # motor de /clima: validaciones y Open-Meteo
|   |   |-- ventanas_015.py            # celdas vano x ventana
|   |   |-- mil_inferencia.py           # lectura del artefacto MIL y sus figuras
|   |   |-- ranking_circuitos.py        # las cuatro bandas de riesgo del circuito
|   |   |-- simulaciones_guardadas.py    # formato de archivo e informe de la simulacion
|   |   |-- almacen_simulaciones.py     # destino local o Volume de Databricks
|   |   |-- report_contract.py          # contrato de /report y /reporte-lote
|   |-- chec_tableros/                  # los cinco tableros
|   |   |-- clima.py                    # nube por vano y clima - 8801
|   |   |-- agrupamiento.py             # agrupamiento de vanos - 8802
|   |   |-- trayectorias_circuitos.py    # trayectorias de circuitos - 8803
|   |   |-- trayectorias_vanos.py        # trayectorias de vanos - 8804
|   |   |-- simulador/                  # "Que pasa si..." - 8866, con kernel de Python
|-- chec_impacto/                       # modelos, entrenamiento e interpretabilidad - 18
|-- aplicaciones/                       # empaquetado de escritorio mac y Windows - 92
|   |-- _comun/                         # menu, servidor, huellas de insumos y empaquetado
|   |-- 00_criticidad_chec/             # el menu CriticidadCHEC - 8800
|   |-- 01_clima/ ... 04_trayectorias_vanos/ # un ejecutable por tablero estatico
|   |-- 06_simulador/                   # el unico que arranca un interprete de Python
|   |-- databricks/                     # el mismo empaquetado, para Databricks Apps
|-- notebooks/
|   |-- 05_mil_vano_ventana.ipynb        # entrena el modelo; se genera desde scripts/
|-- .claude/                             # el contrato de los agentes - 43 archivos
|   |-- skills/                         # /report - /reporte-lote - /informe-gerencial
|   |-- agents/                         # historical - inference - expert-alignment
|   |-- commands/                       # /subir-a-databricks - /actualizar - /instalar-local
|-- .opencode/ - .github/                # los mismos comandos, portados a otros entornos
|-- data/                                # insumos: Indicadores_vano_v3.csv, GEO/, models/
|-- reports/                             # salidas: informes, boveda, paneles y bitacoras
|-- tests/                               # la suite del proyecto - 182 archivos
|-- scripts/                             # generadores, diagnostico y empaquetado
|-- docs/                                # guias de flujo, requisitos y portabilidad
|-- site/                                # el sitio publico - Astro

```

Tres convenciones estructurales condicionan cualquier modificación del repositorio:

- **El cuaderno versionado es salida generada.** Lo escribe un guion de `scripts/`, por lo que las modificaciones aplicadas directamente sobre el archivo `.ipynb` se sobrescriben en la siguiente generación.
- **`.claude/` contiene artefactos ejecutables.** Las habilidades, los roles de agente y los comandos constituyen el contrato ejecutable de los informes y del despliegue. Los directorios `.opencode/` y `.github/` son espejos generados por un guion, de modo que un comando renombrado o retirado se propaga en una sola ejecución.
- **`data/` es entrada y `reports/` es salida.** Esa separación es la que permite que cada aplicación calcule la huella de sus insumos y se reconstruya por sí sola.

6. Tableros de visualización y ampliación de datos climáticos

Los cuatro visores comparten la ventana de análisis — del 1 de noviembre de 2025 al 30 de abril de 2026, **con 159.470 registros vano-evento** sobre 208 circuitos y 27.390 vanos — y los mismos cortes de criticidad, fijos e iguales para todos los circuitos. Esa uniformidad es la que permite comparar dos circuitos sin recalcular umbrales.

Precisión sobre qué cuenta un registro. Los 159.470 registros **no son 159.470 interrupciones**. Cada fila de `Indicadores_vano_v3.csv` corresponde a un **vano señalado dentro del tramo afectado por una interrupción**: un vano que pudo originar el evento y al que, por esa razón, se le atribuye una parte de la afectación. La columna `PORC_APORTE_VANO` guarda esa participación y $UITI_VANO = UITI \times PORC_APORTE_VANO$ reparte entre los vanos del tramo el indicador de la interrupción completa.

Sobre el mismo archivo, las 159.470 filas se agrupan en 6.455 combinaciones distintas de circuito, fecha y equipo de protección; es decir, del orden de seis mil quinientas interrupciones repartidas sobre 27.390 vanos. La lectura correcta de una fila es entonces “este vano estuvo implicado en una interrupción y le corresponde esta fracción del UITI”, y no “aquí ocurrió un evento”. En el resto del informe se usa el término **registro vano-evento** para nombrar esa unidad.

6.1. Nube por vano y clima

Este tablero caracteriza el comportamiento de las variables climáticas sobre los vanos de un circuito y su correspondencia temporal con el UITI observado.

- Mapa del circuito con el UITI diario por vano, coloreado por los cortes fijos de criticidad — Bajo, Medio, Medio-Alto y Alto —, con transformadores e interruptores superpuestos.
- Nube de la variable climática seleccionada sobre el mapa, con escala propia: precipitación, temperatura, viento, ráfaga, descargas a tierra o riesgo por vegetación.
- Dos deslizadores: el día, restringido a los que registraron eventos, y las horas previas al evento, que permiten observar la condición antecedente y no únicamente la del instante.
- Serie de doble eje que enfrenta el UITI del día contra la mediana de la variable seleccionada a lo largo de todo el periodo.
- Seis violines de distribución sobre las doce horas previas: lluvia y temperatura, viento y ráfaga, vegetación y descargas.

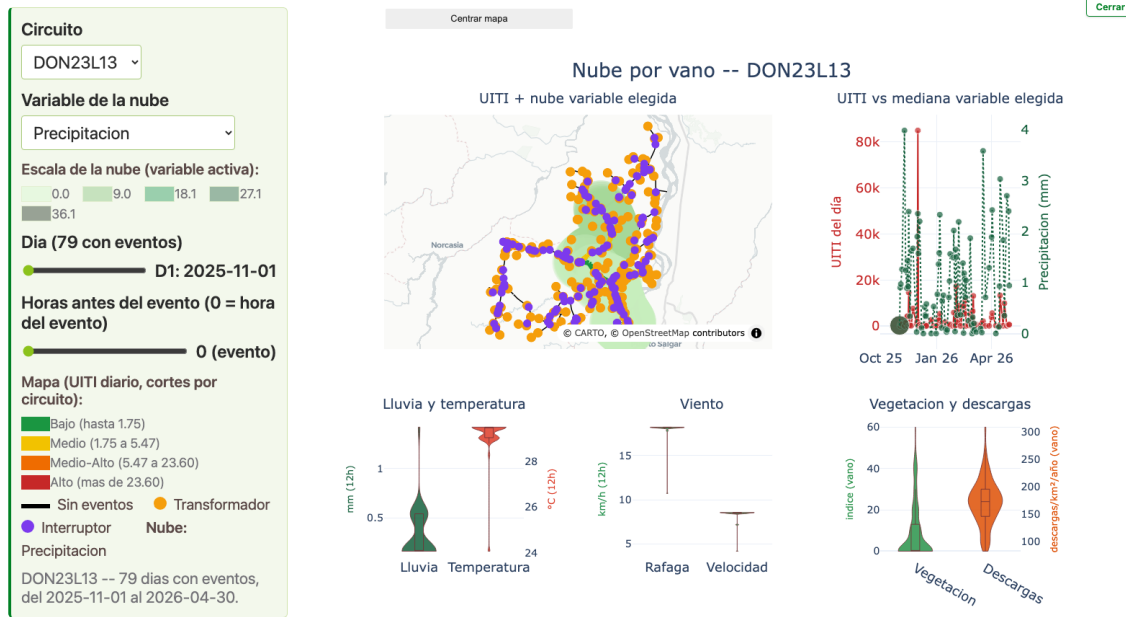


Figura 2: Nube por vano y clima del circuito DON23L13, con 79 días con eventos entre el 1 de noviembre de 2025 y el 30 de abril de 2026. Sobre el mapa, la capa de `UITI_VANO` coloreada por umbrales y la nube de la variable climática del día seleccionado; a la derecha, la serie de doble eje; abajo, los seis violines de distribución.

6.1.1. Captura de datos climáticos con `/clima`

Los datos climáticos se obtienen mediante el comando `/clima`, que consulta el servicio Open-Meteo y ejecuta tres validaciones obligatorias antes de consumir cuota de la interfaz de programación. La clave de la modalidad de pago la introduce el usuario en el momento, permanece únicamente en memoria y no se almacena ni se registra en ninguna bitácora.

Tabla 6: Compuertas de ejecución del comando `/clima`

Paso	Aspecto que resuelve	Detalle
Modo A o B	Destino de la salida	El modo A reescribe <code>data/Indicadores_vano_v3.csv</code> con las coordenadas <code>X1/Y1</code> . El modo B toma cualquier tabla de <code>data/</code> con <code>XPOS1_RED_BASE/YPOS1_RED_AFECTA</code> y deja un archivo nuevo.
Validación 1	Ubicaciones	Se ejecuta en primer lugar y es obligatoria: el proceso no continúa si no se dispone de coordenadas válidas.
Validación 2	Modalidad gratuita o de pago	La modalidad gratuita no requiere credenciales. La de pago solicita la clave al usuario en el momento; no se persiste.
Validación 3	Límites de cuota	Compara las llamadas necesarias contra el presupuesto disponible antes de iniciar la descarga. Si la cuota es insuficiente, el proceso se detiene e informa la condición en lugar de agotarla parcialmente.
Salida	Ubicación del resultado	Modo A: el mismo archivo <code>v3</code> . Modo B: <code>data/clima_<fuente>_<inicio>_a_<fin>.csv</code> . El registro de consumo se escribe aparte, en <code>.clima_cache/</code> .
Unificación	Concatenar ejecuciones	Únicamente entre archivos con el mismo identificador de origen: combinar orígenes distintos produciría una tabla estructuralmente válida y semánticamente inconsistente.

6.2. Agrupamiento de vanos

Este tablero cuantifica en qué vanos se concentra la criticidad de la red. Ajusta un agrupamiento K -medias con $k = 4$ sobre el par (UITI acumulado, número de eventos) de cada vano, y nombra los grupos según el orden de la mediana del UITI — nunca según el índice que devuelve el algoritmo, que cambia entre ejecuciones.

El rango de fechas es ajustable y se redondea a meses completos; las etiquetas resultantes se descargan a una hoja de cálculo. Sobre la ventana de referencia, el reparto es de 4.189 vanos en Bajo (15,3 %), 11.449 en Medio (41,8 %), 10.314 en Medio-Alto (37,7 %) y 1.438 en Alto (5,3 %).



Figura 3: Agrupamiento de vanos por criticidad: 27.390 vanos con eventos entre noviembre de 2025 y abril de 2026, agrupados mediante K -medias con $k = 4$ sobre el plano UITI acumulado contra número de eventos.

6.3. Trayectorias de circuitos

Este tablero establece si un circuito mejora, se deteriora o se mantiene a lo largo del periodo, y en qué ventana ocurre el cambio. Cada punto de la dispersión es un par circuito $\times$ ventana, no un circuito: el mismo circuito aparece tantas veces como ventanas registre, lo que permite observar su desplazamiento en el tiempo.

- Ventana deslizante sobre las once ventanas del periodo, con sus fechas exactas a la vista.
- Mapa del circuito con el UITI del vano en la ventana activa, con los mismos cortes fijos.
- Trayectoria del circuito coloreada por el grupo de cada momento, que hace visible cuándo salta de grupo y cuándo retorna.
- Muestras, UITI y eventos por grupo, para leer la trayectoria contra la población completa.

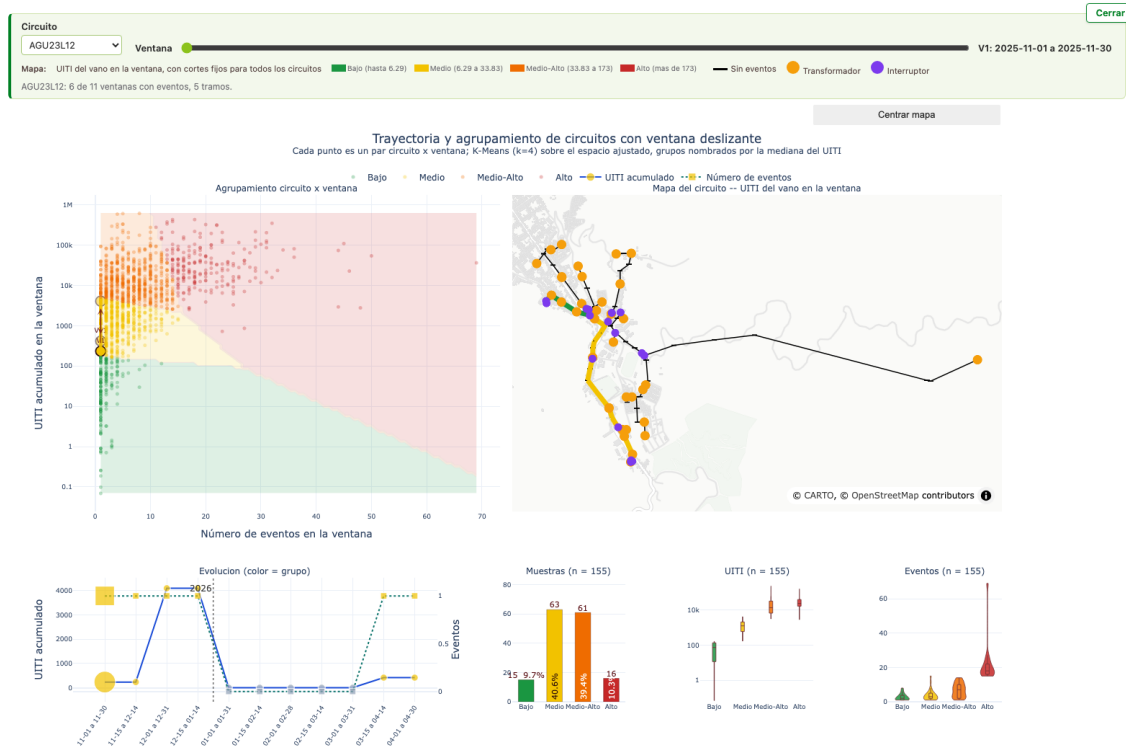


Figura 4: Trayectorias del circuito AGU23L12, con 6 de sus 11 ventanas con eventos. La dispersión sitúa cada par circuito × ventana sobre las cuatro regiones de criticidad; el mapa georreferencia el circuito en la ventana activa.

6.4. Trayectorias de vanos

Este tablero traslada el análisis anterior a la unidad de intervención: la evolución de cada vano ventana a ventana. El agrupamiento *K*-medias se ajusta una sola vez sobre la totalidad de las celdas, de modo que los cortes no se recalculan al cambiar de circuito y dos circuitos resultan comparables.

- Selección de vanos mediante casillas, con acciones de marcar y desmarcar en bloque; la selección se aplica de forma simultánea a todos los paneles.
- Perfil del circuito, que informa cuánto del UITI del periodo concentran los vanos marcados. En el ejemplo de la Figura 5, 15 de 47 vanos explican el 88,3 %.
- Evolución por vano, con UITI y número de eventos superpuestos ventana a ventana.

Este tablero no es únicamente un visor: la geometría de agrupamiento que produce es la que consume después el cálculo de criticidad del modelo y del simulador, y los tres flujos que dependen de ella la reutilizan en modo de solo lectura y verifican su firma antes de usarla.

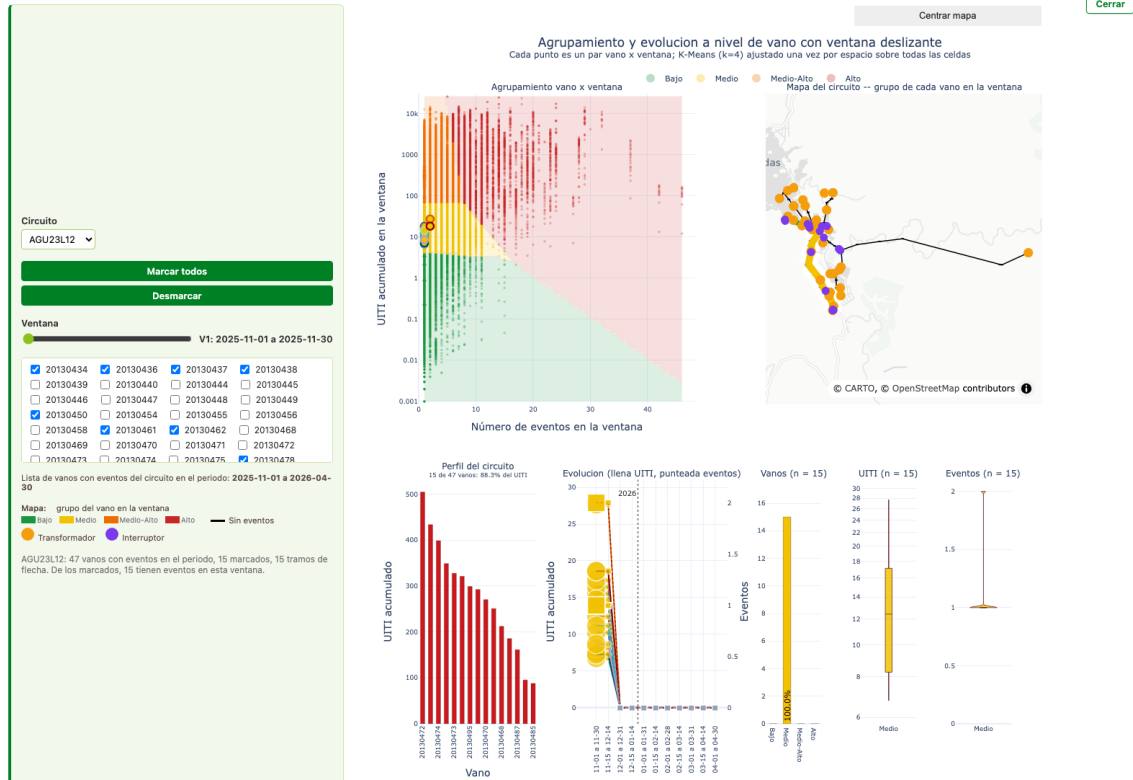


Figura 5: Trayectorias de vanos del circuito AGU23L12, con 15 de 47 vanos seleccionados que concentran el 88,3 % del UITI. Se presentan el panel de selección, el perfil del circuito y la evolución por vano.

6.5. Del vano al grupo de circuito

La etiqueta de criticidad de un circuito no procede de un umbral definido manualmente. Se obtiene en cuatro pasos encadenados, y la Figura 6 los resume.

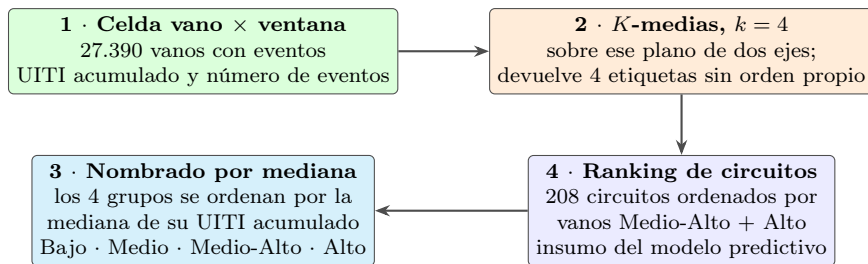


Figura 6: Cadena que convierte el comportamiento observado de un vano en la banda de riesgo de su circuito. El paso 3 corrige la arbitrariedad del índice que devuelve K -medias; el paso 4 sustituye el promedio por un conteo.

Dos criterios explícitos sostienen esa cadena:

- **Nombrado por mediana.** El algoritmo K -medias numera sus grupos con un orden que cambia entre ejecuciones. El nombre se asigna posteriormente a partir de la mediana del UITI, que sí constituye un criterio estable.
- **Conteo en lugar de promedio.** El promedio diluye el efecto de un número reducido de vanos críticos en circuitos con gran cantidad de vanos de baja criticidad. La clasificación contabiliza cuántos vanos caen en Medio-Alto y Alto, que es la carga efectiva.

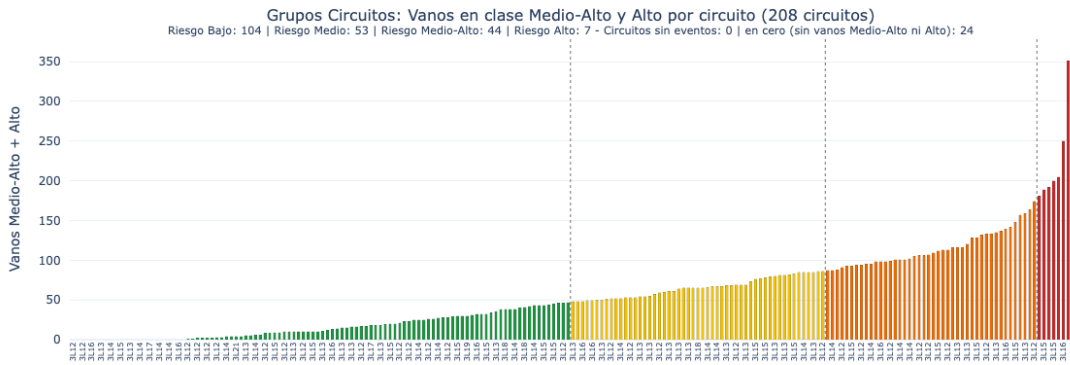


Figura 7: Clasificación de los 208 circuitos por número de vanos Medio-Alto y Alto, con los cortes de las cuatro bandas de riesgo. Las etiquetas del eje horizontal son códigos de circuito y a esta escala operan como referencia de densidad; el valor individual se consulta en el tablero interactivo.

Tabla 7: Reparto de los 208 circuitos por banda de riesgo

Banda	Circuitos	Criterio
Bajo	104	Menor número de vanos en clase Medio-Alto y Alto.
Medio	53	
Medio-Alto	44	
Alto	7	
Sin vanos críticos	24	Mayor concentración de vanos críticos.
		Circuitos sin ningún vano en Medio-Alto ni en Alto.

Un solo vocabulario de grupo. Durante el desarrollo se identificaron dos criterios para definir el grupo de un circuito: el resultado directo de K -medias sobre los circuitos y la clasificación por número de vanos críticos. Para la categoría de riesgo alto, ambos criterios coincidían únicamente en tres circuitos. Se adoptó como criterio definitivo la clasificación por número de vanos Medio-Alto y Alto, y el primero se retiró del código, de modo que un mismo nombre no designe dos particiones distintas.

Las ventanas se traslapan. Cada vano aparece una vez por ventana, lo que convierte el agrupamiento estático en una trayectoria temporal. Las once ventanas se solapan por construcción, de modo que el **UITI no se suma entre ellas**: hacerlo contabilizaría varias veces el mismo registro.

Cómo se construyen las once ventanas. La regla es de dos líneas y no tiene ningún parámetro que ajustar. Sobre el período de análisis — del 1 de noviembre de 2025 al 30 de abril de 2026 — cada mes calendario aporta **dos** ventanas:

- la del **mes completo**, del día 1 al último día del mes; y
- una **ventana cruzada**, que arranca el día 15 de ese mes y termina el día 14 del mes siguiente.

Los seis meses del período producen así doce cortes, de los cuales se conservan los once que caben enteros dentro del período. El resultado son ventanas de entre 28 y 31 días — la duración del mes calendario correspondiente — y no de exactamente treinta.

El propósito de la ventana cruzada es que ningún episodio quede partido por el borde del calendario. Un vano que falla el 28 de diciembre y vuelve a fallar el 4 de enero no comparte ningún mes calendario, pero sí comparte la ventana V4 (15 dic – 14 ene): sin esa ventana

cruzada, ese comportamiento consecutivo se vería como dos episodios aislados en dos meses distintos.

La consecuencia directa de la regla es que cada día del período pertenece a dos ventanas, y por lo tanto **un mismo registro vano-evento entra en dos bolsas**. La Figura 8 lo muestra: las zonas de intersección son las franjas de quince días donde una ventana de mes y una ventana cruzada se superponen. Ese solape es la razón por la que el UITI de un vano nunca se suma a lo largo de las once ventanas — se contaría dos veces el mismo registro — y también la razón por la que la validación cruzada agrupa por vano: sin ese control, un mismo registro aparecería a los dos lados de la partición entre pliegues (Sección 7.7).

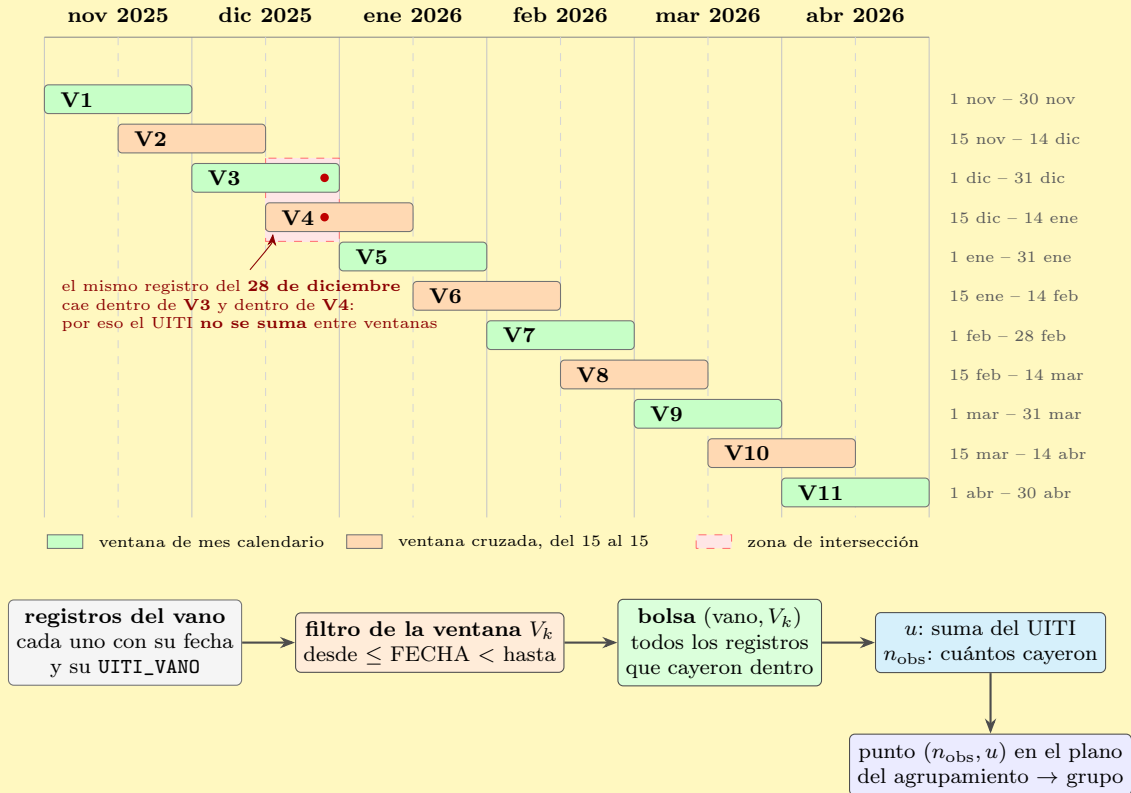


Figura 8: Arriba: las once ventanas del período sobre los seis meses estudiados. Las verdes son meses calendario y las naranjas son las ventanas cruzadas del 15 al 15; cada par consecutivo comparte una franja de quince días, que es la zona de intersección resaltada. Abajo: cómo se genera una bolsa de eventos. Los registros de un vano se filtran por el intervalo de una ventana; los que caen dentro forman la bolsa de esa celda vano $\times$ ventana, cuyo UITI acumulado y número de registros son las dos coordenadas con las que se le asigna el grupo de criticidad.

7. Modelo de inteligencia artificial predictiva M-GCECDL

El módulo predictivo se denomina M-GCECDL, por *Multimodal-Graph Connectivity Enhanced and Conditioned Deep Learning*: un modelo de estimación de criticidad regularizado por restricciones físicas y por un autocodificador del vector de características, construido desde el análisis temporal de eventos sobre vanos mediante una representación por bolsas y múltiples instancias.

7.1. Unidad de análisis: la bolsa vano $\times$ ventana

El modelo no puntúa un vano ni un evento: puntúa una **bolsa**. Una bolsa corresponde a un par vano $\times$ ventana y contiene todas las instancias observadas de ese vano en esa ventana. El

enfoque es de aprendizaje por instancias múltiples: la etiqueta existe a nivel de bolsa — el UITI acumulado y su grupo de criticidad —, mientras que las variables se observan a nivel de instancia.

El preprocesamiento separa `UITI_VANO` como objetivo del cual se derivan las clases, lo que impide su uso como predictor. Las clases son cuatro niveles ordinales — Bajo, Medio, Medio-Alto y Alto — y se generan por agrupamiento (*clustering* con *K-medias*) sobre el plano de dos ejes descrito en la Sección 6.5: el número de eventos observados del vano en el eje horizontal y su UITI acumulado en el eje vertical. Cada celda vano $\times$ ventana cae en uno de los cuatro grupos según a cuál de los cuatro centros queda más cerca en ese plano.

Ese mapa de grupos se calcula una sola vez y queda guardado dentro del archivo del modelo. Al arrancar, el sistema compara el mapa que va a usar con el que tiene guardado; si no coinciden, se detiene con un error explícito en lugar de mover las clases sin dejar registro. El propósito de ese control es sencillo: garantizar que un vano no cambie de grupo por un recálculo silencioso del mapa, sino únicamente porque cambió su comportamiento.

El artefacto entrenado es un único archivo, `data/models/mil_vano_ventana_v1.pt` de 691 KB, que transporta los pesos, los nombres de las 80 variables, la partición en modalidades, el grafo de restricciones físicas proyectado, la geometría de agrupamiento y el desglose de desempeño por clase. Esa autocontención permite que el visor del cuaderno, el simulador y las aplicaciones lean el mismo modelo sin reconstruir nada.

7.2. Arquitectura del modelo

El modelo predice un escalar por bolsa, $\hat{p}_b \approx \log(1 + u_b)$, y deriva la clase con la regla de vecino más cercano. La Figura 9 presenta su diagrama de bloques.

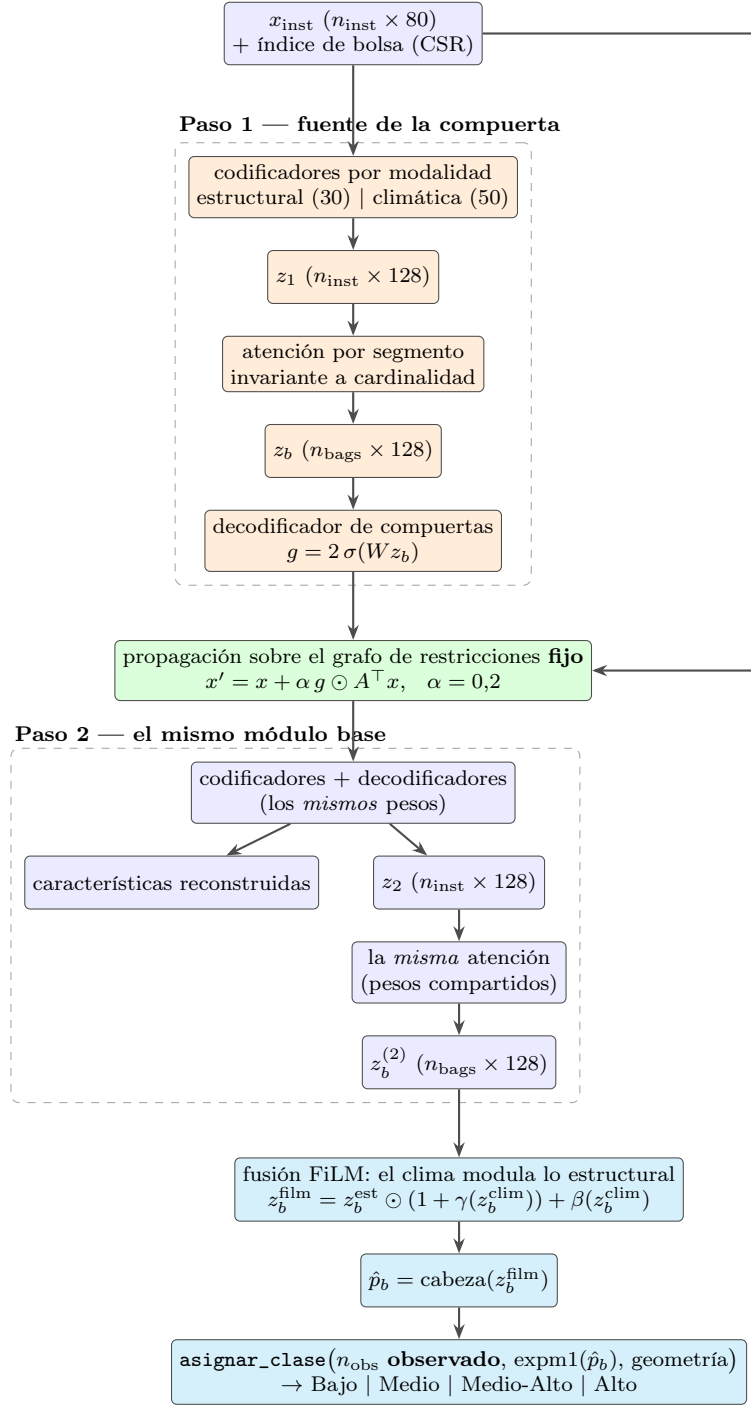


Figura 9: Diagrama de bloques del modelo por bolsas. La primera pasada produce la compuerta por bolsa que modula el grafo de restricciones fijo; la segunda recodifica la entrada ya propagada reutilizando exactamente los mismos pesos. La clase no procede de la red, sino de asignarse al centroide o región correspondiente de la geometría de agrupamiento congelada.

La figura anterior, explicada paso a paso. El diagrama se lee de arriba hacia abajo y describe una sola pasada de cálculo sobre una bolsa. En lenguaje corriente:

1. **Entrada.** Llegan todos los registros de la bolsa. Cada registro es una fila de 80 números que describe el vano, su transformador, su topología, la causa del evento y el clima de los días previos. El “índice de bolsa” es simplemente la lista que dice qué filas pertenecen a qué vano-ventana.

2. **Paso 1: mirar la bolsa y decidir cuánto pesa el contexto.** Los datos se comprimen por separado en dos bloques — lo *estructural* (30 variables) y lo *climático* (50 variables) —, un mecanismo de atención resume las filas de la bolsa en un solo vector, y de ese resumen sale una **compuerta**: un número por bolsa, entre 0 y 2, que responde a la pregunta “¿cuánto debe pesar en esta bolsa lo que le ocurre a las variables vecinas?”. La compuerta no cambia las relaciones; sólo les sube o les baja el volumen.
3. **Propagación sobre el grafo de restricciones.** El grafo es un mapa fijo de 156 relaciones entre variables, escrito a mano a partir del criterio de ingeniería — por ejemplo, que la altura del apoyo y la longitud de la cruceta se relacionan con el riesgo por vegetación. Cada variable recibe un aporte de las variables con las que el grafo dice que está relacionada, en la proporción que fija la compuerta. Ese mapa **no se aprende**: el modelo no puede inventar una relación que la ingeniería no haya declarado.
4. **Paso 2: volver a mirar, ya con el contexto incorporado.** Los datos, ahora enriquecidos por el grafo, pasan de nuevo **por los mismos bloques y con los mismos pesos** del paso 1 — no hay una segunda red. Esa reutilización es deliberada: obliga a que la representación sirva tanto para los datos crudos como para los datos propagados. En paralelo, el modelo reconstruye la entrada a partir de esa representación comprimida, lo que impide que descarte información por el camino.
5. **Fusión: el clima modula lo estructural.** En vez de sumar los dos bloques, el clima ajusta la lectura de lo estructural: escala y desplaza el vector estructural según las condiciones meteorológicas de la ventana. Es la forma matemática de decir que un mismo apoyo, con el mismo estado, no tiene el mismo riesgo en un mes seco que en un mes de lluvias intensas.
6. **Salida.** La red entrega *un solo número*: el UITI acumulado que estima para esa bolsa. **No entrega una clase.** La clase se obtiene después, fuera de la red, ubicando el punto (número de eventos observado, UITI estimado) en el mapa de agrupamiento congelado y viendo en qué grupo cae. Por eso el número de eventos es siempre un dato observado y nunca una predicción del modelo.

El diseño garantiza tres propiedades con consecuencias directas sobre la validez de la clasificación:

- **Invarianza de cardinalidad.** Duplicar todas las instancias de una bolsa deja $\hat{p}_b$ idéntico. En ausencia de esta garantía el modelo podría acceder al número de eventos por una vía indirecta, siendo esa cantidad una de las dos coordenadas que definen la clase.
- n_{obs} **es observado, nunca predicho.** De las dos coordenadas que determinan la clase, el modelo solo aporta u .
- **El grafo es fijo.** Se registra como memoria intermedia y no como parámetro, de modo que ninguna arista se aprende. El único elemento ajustable en todo el camino del grafo es la compuerta g_b , que escala por bolsa las aristas existentes sin crear ni eliminar ninguna.

7.3. Variables de entrada

El modelo opera sobre una matriz de 80 columnas por instancia, repartidas en dos modalidades: 30 estructurales — vano, transformador, topología y las diez columnas derivadas de COD_CAUSA, es decir su frecuencia relativa y nueve indicadores — y 50 climáticas — riesgo por vegetación, descargas a tierra y cuatro series meteorológicas con doce rezagos cada una.

Tabla 8: Familias de variable que ingresan al modelo

Familia	Tipo	Ejemplos
Estructural del vano	Numérica	ALTURA, CNT_FASES, LONG_CRUCETA, VAL_CRIT_APOYO, CANTIDAD_TIERRA
Estructural del vano	Categórica	CONDUCTOR, TIPO, CALIBRE_NEUTRO, NG_RED, TIPO_TAX
Transformador	Numérica	CAPACIDAD_NOMINAL, PROMEDIO_KWH_TRF, CNT_TRF
Topología	Numérica	CNT_VN: vanos hasta el equipo que protege el tramo
Entorno	Numérica	NR_T: riesgo por vegetación
Clima	Serie con 12 rezagos	Precipitación, temperatura, viento, ráfaga y descargas a tierra
Evento	Categórica codificada	COD_CAUSA, su frecuencia relativa y nueve indicadores
Espacio y tiempo	Coordenada y fecha	X2, Y2, FECHA_OPERACION_VANO
Objetivo	Numérica por bolsa	UITI_VANO acumulado en la ventana

7.4. Formulación matemática

Agregación de la bolsa. La atención opera por segmento, es decir una distribución por bolsa y no por lote:

$$e_i = w^\top \tanh(Vz_i), \quad a_i = \frac{\exp(e_i)}{\sum_{j \in b(i)} \exp(e_j)}, \quad z_b = \sum_{i \in b} a_i z_i. \quad (1)$$

Propagación por el grafo. La compuerta por bolsa $g_b = 2\sigma(Wz_b) \in (0, 2)^E$ modula el grafo fijo y escribe únicamente en las columnas destino de las E aristas:

$$x'_i = x_i + \alpha \sum_{(r \rightarrow c) \in \mathcal{E}} g_{b(i), rc} A_{rc} x_{i,r} \mathbf{e}_c. \quad (2)$$

Función de costo. Sobre un lote de B bolsas, con $t_b = \log(1 + u_b)$ el objetivo de la bolsa b , la pérdida activa reúne cuatro términos:

$$\mathcal{L} = \lambda_{\text{sup}} \mathcal{L}_{\text{sup}} + \lambda_{\text{rec}} \tilde{\mathcal{L}}_{\text{rec}} + \lambda_{\text{MI}} \mathcal{L}_{\text{MI}} + \lambda_{\text{cl}} \mathcal{L}_{\text{cl}}. \quad (3)$$

El término supervisado es un error cuadrático ponderado por la densidad inversa del objetivo, lo que evita que la cola alta de UITI quede dominada por la masa central de la distribución:

$$\mathcal{L}_{\text{sup}} = \frac{1}{B} \sum_b \tilde{w}(t_b) (\hat{p}_b - t_b)^2, \quad w(t) = \frac{1}{\max(\hat{f}_{\text{KDE}}(t), \varepsilon)}, \quad \tilde{w} = \frac{w}{\bar{w}}. \quad (4)$$

La densidad $\hat{f}_{\text{KDE}}$ se ajusta una sola vez sobre el pliegue de entrenamiento — ajuste realizado exclusivamente con datos del pliegue de entrenamiento, para evitar fuga de información — y los pesos se renormalizan a media unitaria por lote, de manera que la escala resulte comparable con un error cuadrático plano.

El término de reconstrucción se calcula contra la entrada estandarizada $z = (x - \mu)/\sigma$ y se acota sin anular el gradiente:

$$\mathcal{L}_{\text{rec}}^{\text{raw}} = \frac{1}{np} \sum_{i,j} (\hat{x}_{ij} - z_{ij})^2, \quad \tilde{\mathcal{L}}_{\text{rec}} = \frac{\mathcal{L}_{\text{rec}}^{\text{raw}}}{1 + \mathcal{L}_{\text{rec}}^{\text{raw}}} \in [0, 1). \quad (5)$$

Su objetivo es el x original y no x' . De emplearse x' , la compuerta — único elemento ajustable del camino del grafo — controlaría su propio objetivo y podría reducir la pérdida simplificándolo, sin mejorar la representación.

El término de información mutua es el único que ata la representación aprendida a la estructura declarada. Compara, mediante información mutua cuadrática de Rényi, un núcleo gaussiano sobre los perfiles de variables reconstruidas contra el núcleo del grafo fijo:

$$\mathcal{L}_{\text{MI}} = 1 - \bar{I}_2(K_r, K_g), \quad \bar{I}_2 = \frac{I_2}{\log p}. \quad (6)$$

K_g no se estima de los datos: se construye una sola vez en el constructor de la pérdida a partir de la lista de nombres de las variables, se almacena como constante y no recibe gradiente. El lado declarado es invariante por construcción, de modo que el término solo puede aproximar la representación aprendida a la estructura del grafo, y no el grafo a los datos.

Finalmente, el término de clase es una entropía cruzada sobre las fronteras de la geometría congelada, diferenciable a través de $\hat{p}_b$:

$$\hat{u}_b = \text{softplus}(\text{expm1}(\hat{p}_b)), \quad \mathcal{L}_{\text{cl}} = \text{CE}\left(-\frac{d^2(n_b^{\text{obs}}, \hat{u}_b)}{T}, c_b^*\right), \quad T = 0,01. \quad (7)$$

Sin este término, ningún componente del costo incorporaría la posición de las fronteras entre centroides, que es precisamente lo que evalúa la métrica de desempeño. La temperatura no hereda el valor unitario de otros usos del proyecto: con distancias cuadradas de mediana 0,038, una temperatura unitaria deja la distribución *softmax* prácticamente uniforme — entropía de 1,3850 contra $\ln 4 = 1,3863$ — y el término queda en su piso desde la primera época. La clase objetivo c_b^* se deriva de lo observado con la misma geometría y no se recibe por parámetro, lo que impide introducir un objetivo inconsistente con la regla de asignación.

Tabla 9: Pesos del artefacto entrenado y participación del grafo fijo en cada término

Término	Peso	Usa el grafo	Papel
Supervisado	1,00	Indirecto	No aparece en la fórmula; entra porque $\hat{p}_b$ se calcula sobre x' .
Reconstrucción	0,01	No	Deliberado: el objetivo es el x original.
Información mutua	0,01	Sí, como referencia	Único término que ata la representación a la estructura declarada.
Clase	1,00	No	Usa la geometría de centroides, artefacto fijo distinto del grafo.
Desviación de compuertas	0,00	Sí, como ancla	Penalizaría apartarse del grafo tal como fue declarado. Desactivado.
Supervisión por modalidad	0,00	No	Inerte bajo la fusión FiLM; solo resulta pertinente bajo fusión por confiabilidad.

Corresponde registrar una limitación de orden cuantitativo y no conceptual: los dos términos del grafo están acotados en $[0, 1]$ y pesan 0,01 cada uno, por lo que su contribución conjunta al costo total no supera 0,02, frente a un término supervisado cuyo rango observado fue de 0,35 a 6,8. Con esta configuración la participación de esos dos *términos* en el gradiente es marginal, y su aporte efectivo requiere una ablación explícita para ser establecido. **Conviene distinguirlo con precisión:** el grafo actúa sobre el modelo por dos vías, y solo una de ellas es este término del costo. La otra es la **propagación** — $x' = x + \alpha g \odot A^\top x$ —, que es estructural, opera en cada pasada y no depende de ningún peso de penalización. Es esa vía la que impide que el modelo utilice asociaciones no declaradas, y es la que sostiene la garantía de interpretabilidad. La curva plana de la Figura 11 corresponde al término del costo, no a la propagación.

Los hiperparámetros del artefacto vigente son: dimensión oculta 128, dimensión de embebido 64, dimensión de atención 64, *dropout* 0,1, $\alpha = 0,2$, tasa de aprendizaje 10^{-3} , decaimiento de pesos 10^{-5} , lotes de 256 bolsas, 30 épocas y 5 pliegues, con semilla fija.

7.5. Grafo de restricciones físicas

Las relaciones entre variables no se aprenden de forma autónoma: se declaran previamente mediante un grafo de restricciones físicas, definido en código por la función `construir_aristas_grafo_chec` y almacenado dentro del artefacto `.pt`. Esa ubicación garantiza que el simulador, los informes y el cuaderno operen sobre exactamente la misma estructura.

El grafo declarado comprende **156 relaciones dirigidas sobre 156 variables** y es acíclico: todo camino avanza hacia `UITI_VANO`, que es su único destino final. Sus pesos son juicio experto declarado variable por variable — valores como 0,60, 0,75 o 0,90 — y no correlaciones estimadas de los datos. Lo único que los datos determinan es qué nodos existen: si un código de causa no alcanza el umbral de frecuencia relativa del 1 %, su columna no está y las aristas que lo tocaban no se proyectan.

Su composición, verificada sobre el código vigente, es la siguiente:

Tabla 10: Composición de las 156 relaciones declaradas del grafo

Clase de relación	Cantidad	Descripción
Cadena de rezago climático	99	Nueve familias meteorológicas $\times$ once enlaces entre rezagos consecutivos, con peso 0,90.
Rezago cero hacia la causa	9	Cada familia en la hora del evento alimenta <code>COD_CAUSA</code> , con peso 0,85.
Ráfaga hacia riesgo vegetal	1	<code>wind_gust_spd_0</code> $\rightarrow$ <code>NR_T</code> , con peso 0,80.
Relaciones de dominio	47	Recorren el camino desde la geometría del vano hasta el cálculo de <code>UITI_VANO</code> .
Total declarado	156	

La selección de variables elimina después una parte de esos nodos. En lugar de descartar las aristas que tocaban un nodo eliminado, un recorrido en anchura busca, desde cada nodo conservado, el primer nodo también conservado que sea alcanzable a través de los eliminados, y registra una *arista virtual* con el peso mínimo del trayecto. El mínimo es la elección conservadora: una cadena no puede declararse más fuerte que su eslabón más débil. Sobre el artefacto vigente el resultado es una matriz de adyacencia de 80×80 con **64 aristas no nulas**.

7.5.1. Alcance de cada modo dentro del grafo

Las variables se agrupan en seis modos, y el grafo define qué relaciones se permiten entre ellos. La tercera columna de la Tabla 11 no constituye interpretación libre: recoge lo que el análisis de criticidad del proyecto declara que se busca en cada relación, y es también el marco con el que los agentes leen el modelo.

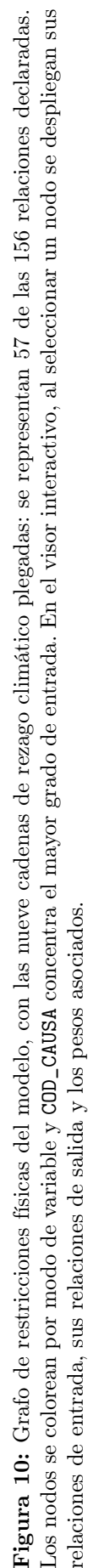


Tabla 11: Modos de variable, conexiones fijadas y relaciones evaluadas

Modo	Conexiones fijadas en el grafo	Relaciones y posibles causas evaluadas
Entorno / Riesgo	Hacia el mismo modo (100) y hacia Evento / Impacto (11).	Acumulación del estrés ambiental en las doce horas previas: cada rezago apunta al siguiente, de modo que la cadena representa la condición antecedente y no el instante del evento. Es la vía por la que el modelo puede sostener una hipótesis ambiental, nunca afirmarla.
Físicas / Eléctricas	Hacia el mismo modo (3), Evento / Impacto (2), Entorno / Riesgo (1) y Topología (1).	Características constructivas que se explican entre sí y que modulan la susceptibilidad del tramo: conductor, longitud y calibre del neutro. La ausencia de cable de guarda o neutro incrementa la exposición a las descargas.
Activos	Hacia el mismo modo (11), Entorno / Riesgo (3) y Evento / Impacto (1).	Encadenan apoyo, transformador y sus atributos para identificar el activo afectado, y cuantifican la carga servida aguas abajo. Estas variables explican la magnitud del impacto, no su frecuencia.
Topología	Hacia el mismo modo (6), Protección (3) y Evento / Impacto (1).	Ubican el tramo y determinan el alcance del equipo que lo protege — número de vanos que cubre y distancia —, lo que fija el alcance de una interrupción, y establecen la fracción del impacto del circuito que corresponde al vano.
Protección	Hacia el mismo modo (4) y Evento / Impacto (2).	Correspondencia entre vano y equipo de protección, y efecto del tipo de equipo sobre la duración de la interrupción y el número de usuarios afectados aguas abajo.
Evento / Impacto	Hacia el mismo modo (7).	Descomposición del UITI: duración por usuarios afectados y, a partir de ahí, el aporte del vano.

El grafo restringe las relaciones que el modelo puede utilizar, pero **no establece causalidad**. Una relación fijada indica que el modelo puede emplear esa relación, no que el fenómeno ocurra: la vía de NR_T hacia COD_CAUSA existe para que la vegetación pueda sostener una hipótesis de causa, y esa hipótesis requiere respaldo en los datos observados dentro de la ventana analizada. Por esa razón los informes se expresan en términos de lo que resulta compatible con la evidencia, y no de lo que la causó.

7.6. Aplicación sobre datos nuevos

El procedimiento de inferencia sobre observaciones que el modelo no vio durante el entrenamiento consta de cinco pasos:

1. Se arma la matriz de instancias con el mismo codificador de COD_CAUSA empleado en el entrenamiento.
2. Se carga el artefacto .pt, que aporta los pesos, el grafo de restricciones y la geometría de agrupamiento congelada.
3. Cada fila ingresa como bolsa unitaria con $n_{\text{obs}} = 1$, que es lo correcto al evaluar eventos individuales.
4. La red devuelve $\hat{u}$ y la función de asignación determina la clase a partir de la geometría.
5. El mecanismo de atención proporciona, por bolsa, la contribución relativa de las variables de mayor peso, que constituye el insumo del diagnóstico del simulador.

7.7. Protocolo de validación y resultado

La métrica es macro-F1 fuera de pliegue, con validación cruzada estratificada y agrupada por la clave (CIRCUITO, FID_VANO), de modo que ninguna bolsa de un mismo vano cruce la frontera entre pliegues. Las ventanas se solapan por construcción, así que sin ese control un mismo evento aparecería a ambos lados de la partición. La evaluación se restringe al subconjunto de bolsas cuyo vano tiene al menos dos ventanas y cuya clase observada no es constante entre ellas: deliberadamente, el subconjunto difícil.

El modelo se contrasta contra tres líneas base: predicción mayoritaria, un bosque aleatorio sobre las variables estructurales sin clima, y persistencia de la clase de la ventana anterior.

Tabla 12: Desempeño fuera de pliegue del modelo y de sus tres líneas base, sobre 62.114 bolsas

Brazo	Exactitud	macro-F1
Bosque aleatorio estructural (mejor línea base)	0,8653	0,8812
Modelo M-GCECDL	0,8535	0,8710
Persistencia	0,4484	0,3821
Mayoritaria	0,4384	0,1524

Tabla 13: Desempeño del modelo por grupo de criticidad

Grupo	Precisión	Exhaustividad	F1	Soporte
Bajo	0,8443	0,8252	0,8346	13.323
Medio	0,8295	0,8514	0,8403	27.229
Medio-Alto	0,8571	0,8453	0,8512	15.220
Alto	0,9741	0,9421	0,9578	6.342

Tabla 14: Resultado del protocolo de validación frente al criterio de superioridad prerregistrado

Concepto	Valor
Bolsas evaluadas en validación cruzada agrupada	62.114
macro-F1 del modelo	0,8710
macro-F1 de la mejor línea base	0,8812
Criterio de <i>superioridad</i> prerregistrado	+5,0 puntos
Diferencia observada	−1,02 puntos
Dispersión del entrenamiento entre corridas idénticas	±7 puntos
Lectura	Desempeño equivalente

Cómo debe leerse esta tabla. El criterio prerregistrado de +5,0 puntos era una meta de **superioridad**, no un umbral de funcionamiento: se fijó antes de entrenar para evitar que, una vez vistos los resultados, cualquier mejora marginal se presentara como un avance. Que no se haya alcanzado esa meta **no significa que el modelo no funcione**. La lectura correcta de los números es la siguiente.

La diferencia observada es de −1,02 puntos de macro-F1 sobre 62.114 bolsas, mientras que la propia dispersión del entrenamiento entre dos corridas de configuración idéntica es de siete puntos. Una diferencia de un punto medida con una regla cuya precisión es de siete puntos **no distingue a los dos modelos**: en términos de desempeño, el modelo M-GCECDL y el bosque aleatorio son equivalentes sobre este conjunto. Ambos aciertan alrededor del 86 % de las bolsas, y ambos superan por más de cuarenta puntos a las líneas base de persistencia y de clase mayoritaria, que es la comparación que realmente indica si hay señal aprendible en los

datos.

Con desempeño equivalente, la elección entre uno y otro se decide por lo que cada uno ofrece además del acierto, y ahí la comparación no es cerrada:

- **Respeta las restricciones físicas.** El bosque aleatorio puede apoyarse en cualquier correlación que encuentre. El modelo propuesto solo puede propagar información a través de las 156 relaciones que la ingeniería declaró en el grafo, de modo que una explicación imposible en la red eléctrica es también imposible en el modelo.
- **Respeta la estructura temporal.** La unidad es la bolsa vano $\times$ ventana y la agregación es invariante a la cantidad de registros, de manera que el número de eventos — una de las dos coordenadas que definen la clase — no puede filtrarse como predictor por una vía indirecta. El bosque aleatorio no ofrece esa garantía por construcción.
- **Es más eficiente.** 150.926 parámetros, 691 KB de artefacto y unos diez minutos de reentrenamiento completo en una CPU de dos núcleos, sin GPU.
- **Es interpretable por diseño.** La atención entrega, por bolsa, qué variables pesaron y cuánto; el grafo indica por qué esa relación estaba permitida; y el simulador convierte esa lectura en una recomendación accionable con su costo. Un bosque aleatorio entrega importancias globales de variable, no una explicación por vano y ventana ligada a relaciones físicas declaradas.

Frente a las líneas base habituales en datos tabulares. El punto de referencia natural de un problema como este son los bosques aleatorios y las arquitecturas profundas tabulares del tipo TabNet. El bosque aleatorio está **medido** en este informe (Tabla 12) y el resultado es el ya descrito: desempeño equivalente. Frente a las arquitecturas tabulares profundas, la comparación pertinente en este proyecto es de diseño y no de puntaje: esas arquitecturas aprenden por sí mismas qué combinaciones de variables usar, mientras que aquí ese espacio está deliberadamente restringido al grafo de relaciones físicas declaradas y a la ventana temporal. Se renuncia a una parte de la libertad del modelo a cambio de que cada explicación sea trazable hasta una relación que el personal técnico reconoce, y esa restricción se sostiene sin pagar desempeño.

Qué queda por hacer, y por qué es una tarea acotada. La dispersión de siete puntos entre corridas idénticas proviene del no determinismo de la ejecución y no de la formulación del modelo. Fijar las semillas y el orden de los lotes es un trabajo de ingeniería acotado, y es el paso que permitirá comparar variantes entre sí con confianza. Mientras tanto, el perfil de error ya es el adecuado para el uso previsto: la clase Alto —la que dispara la intervención— es la mejor predicha, con F1 de 0,958, y los errores caen en clases contiguas.

El resultado se reporta junto con los matices que una comparación de una sola cifra no recoge. Ninguna clase queda sin predicciones; la clase Alto registra el mejor desempeño del modelo, con F1 de 0,958, y el error se concentra en clases contiguas — la confusión entre Bajo y Alto ocurre una vez en 13.323 casos —, perfil adecuado para priorización operativa.

La consideración determinante es que **la brecha es menor que el ruido del propio entrenamiento**. Dos corridas de configuración idéntica arrojaron 0,8012 y 0,8710, es decir siete puntos de dispersión atribuibles al no determinismo de la ejecución, mientras que la línea base reprodujo su valor de forma idéntica en seis corridas. Bajo esa dispersión, la diferencia de $-1,02$ puntos no permite concluir que un modelo sea mejor que el otro, y la estabilización del determinismo del entrenamiento constituye el prerrequisito para cualquier comparación fina posterior.

7.8. Cómo se comporta el modelo: curvas, matriz de confusión y un caso completo

Esta sección muestra el comportamiento del modelo con las tres evidencias habituales — las curvas de entrenamiento, las curvas ROC y la matriz de confusión — y cierra con un caso real recorrido de principio a fin. Todas las cifras provienen de mediciones sobre este mismo repositorio; los guiones que producen las dos primeras figuras quedan en `Informe_Impacto_CHEC/scripts/`.

7.8.1. Curvas de entrenamiento: ¿está aprendiendo?

La curva de entrenamiento responde a la pregunta más básica que se le puede hacer a un modelo: mientras se ajusta, ¿su error baja? La Figura 11 lo muestra sobre un pliegue real de 88.987 bolsas de entrenamiento, a lo largo de las 30 pasadas completas por los datos que constituyen el entrenamiento estándar. La escala vertical es logarítmica porque el error se reduce en más de un orden de magnitud y en escala normal el final de la curva sería ilegible.

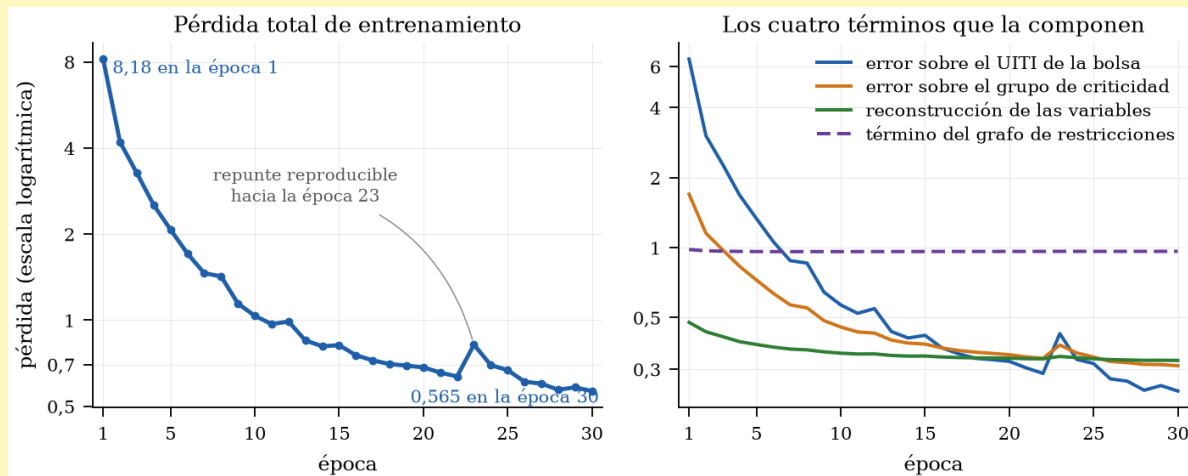


Figura 11: Curvas de entrenamiento medidas sobre el primer pliegue (88.987 bolsas de entrenamiento, 22.246 de prueba), 30 épocas, 103,6 segundos en CPU. A la izquierda, el error total: baja de 8,18 a 0,565, es decir a un catorceavo de su valor inicial. A la derecha, los cuatro términos que lo componen, en la misma escala logarítmica.

Tres lecturas se desprenden de la figura, y conviene tomarlas en orden.

El error baja de forma sostenida. El error total pasa de 8,18 en la primera pasada a 0,565 en la trigésima. La caída es rápida durante las diez primeras épocas y luego se aplanan: a partir de la época 20 las mejoras son pequeñas, lo que indica que treinta pasadas son suficientes y que añadir más no cambiaría el resultado de forma apreciable.

Cada término hace lo que debe. El panel derecho separa el error total en sus cuatro componentes:

- El **error sobre el UTIL de la bolsa** — lo que el modelo realmente predice — es el que más baja: de 6,47 a 0,242. Es el término dominante al principio y el más pequeño al final.
- El **error sobre el grupo de criticidad** baja de 1,70 a 0,311. Es el término que empuja al modelo a que su estimación caiga del lado correcto de las fronteras del agrupamiento, y no solo cerca del valor numérico.

- La **reconstrucción de las variables** baja de 0,477 a 0,327. Su papel es vigilar que la representación interna no descarte información: si el modelo pudiera olvidar variables, este término subiría.
- El **término del grafo de restricciones** permanece prácticamente plano, entre 0,981 y 0,964. Esto no indica que el grafo no actúe: el grafo interviene sobre todo por la vía *estructural* de la propagación, que opera siempre y no depende de este término. Lo que la curva plana señala es que la penalización adicional aporta poco gradiente, y por eso su contribución propia debe establecerse mediante una ablación explícita antes de atribuirle efecto.

Hay un repunte hacia la época 23, y es conocido. La curva no es perfectamente monótona: presenta un pico reproducible cerca de la época 23, tras el cual el descenso se reanuda y termina por debajo del punto previo al repunte. Es un comportamiento documentado del entrenamiento actual y es el mismo fenómeno que explica los siete puntos de dispersión entre corridas idénticas de la Sección 7.7. Fijar ese no determinismo es la tarea de ingeniería pendiente; no afecta a la validez de lo que el modelo aprende, sino a la precisión con la que se pueden comparar dos variantes entre sí.

7.8.2. Curvas ROC: ¿separa bien los vanos críticos de los que no lo son?

La curva ROC responde a una pregunta distinta de la anterior, y más cercana al uso operativo: si se ordenan todos los vanos de mayor a menor riesgo según el modelo y se va bajando el listón, ¿cuántos vanos verdaderamente críticos se capturan por cada falsa alarma que se acepta? Una curva pegada a la esquina superior izquierda es un buen ordenamiento; la diagonal punteada es lo que daría el azar.

El área bajo esa curva (AUC) resume el resultado en un solo número, con una lectura muy concreta: **es la probabilidad de que, tomando al azar un vano crítico y uno que no lo es, el modelo le asigne más riesgo al crítico**. Un AUC de 0,5 es azar puro; uno de 1,0 es un ordenamiento perfecto.

Como el modelo entrega un solo número continuo — el UITI estimado —, y la clase es ordinal, la lectura correcta es acumulativa: se evalúan los tres cortes sucesivos de la escala. La Figura 12 los presenta, con el bosque aleatorio superpuesto para comparar.

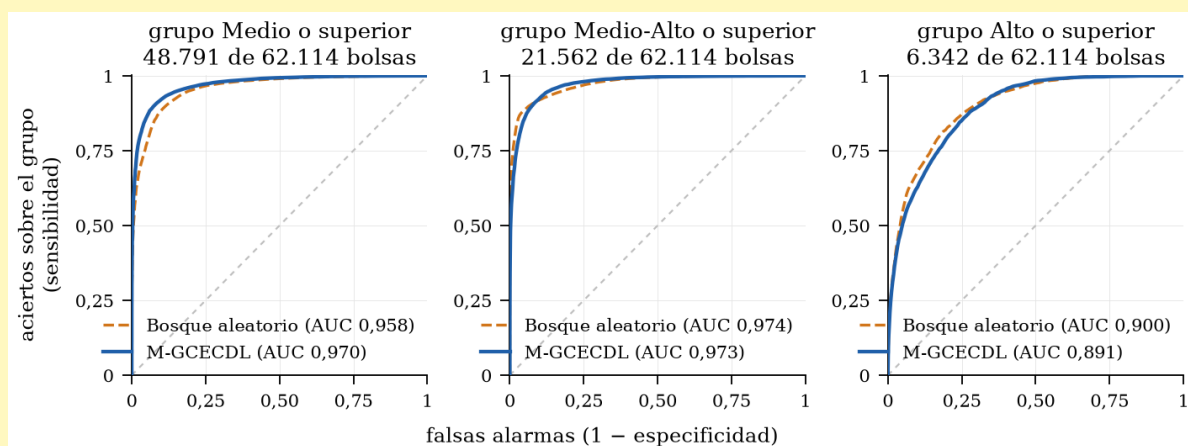


Figura 12: Curvas ROC sobre las 62.114 bolsas evaluadas fuera de pliegue, para los tres cortes de la escala ordinal de criticidad. La curva azul es el modelo M-GCECDL; la naranja punteada, el bosque aleatorio estructural. Las dos se calculan sobre exactamente las mismas bolsas y las mismas predicciones fuera de pliegue que sostienen las tablas anteriores.

Tabla 15: Área bajo la curva ROC por corte de la escala de criticidad

Corte	Bolsas positivas	AUC M-GCECDL	AUC bosque aleatorio
Medio o superior	48.791	0,970	0,958
Medio-Alto o superior	21.562	0,973	0,974
Alto	6.342	0,891	0,900

Los tres valores están entre 0,89 y 0,97, es decir muy por encima del azar: el modelo ordena los vanos por riesgo de forma fiable, que es exactamente lo que necesita una lista de priorización. La comparación con el bosque aleatorio confirma lo dicho en la Sección 7.7 desde otro ángulo: el modelo es mejor en el corte inferior, el bosque lo es marginalmente en los otros dos, y las diferencias — entre uno y doce milésimas — son del orden de la dispersión del propio entrenamiento. **Los dos ordenan los vanos igual de bien.**

El corte más exigente — aislar la clase Alto — es también el de menor AUC, lo cual es esperable: son 6.342 bolsas contra 55.772, y separar una minoría pequeña siempre es más difícil. Aun así, un AUC de 0,891 significa que en casi nueve de cada diez comparaciones el modelo pone por delante al vano que efectivamente resultó Alto.

7.8.3. Matriz de confusión: ¿en qué se equivoca cuando se equivoca?

La matriz de confusión abre el resultado global y muestra, para cada grupo real, en qué grupo lo ubicó el modelo. Se lee por filas: cada fila es un grupo observado y se reparte entre las cuatro columnas de predicción. La diagonal son los aciertos.

Tabla 16: Matriz de confusión del modelo sobre las 62.114 bolsas evaluadas fuera de pliegue. Filas: grupo observado. Columnas: grupo predicho. Entre paréntesis, el porcentaje de la fila.

Observado	Grupo predicho por el modelo				Total
	Bajo	Medio	Medio-Alto	Alto	
Bajo	10.994 (82,52 %)	2.308 (17,32 %)	20 (0,15 %)	1 (0,01 %)	13.323
Medio	2.007 (7,37 %)	23.182 (85,14 %)	1.986 (7,29 %)	54 (0,20 %)	27.229
Medio-Alto	18 (0,12 %)	2.232 (14,66 %)	12.866 (84,53 %)	104 (0,68 %)	15.220
Alto	3 (0,05 %)	225 (3,55 %)	139 (2,19 %)	5.975 (94,21 %)	6.342

Lo que esta tabla dice, en orden de importancia operativa:

- **El grupo Alto es el mejor predicho.** De 6.342 bolsas realmente Altas, el modelo identifica 5.975, el 94,21 %. Es el grupo que dispara la intervención, y es donde el modelo acierta más. Solo tres bolsas Altas terminaron clasificadas como Bajas.
- **Prácticamente no hay errores graves.** Sumando la diagonal y las dos casillas contiguas a cada lado, el modelo cae en el grupo correcto o en uno adyacente en 61.793 de las 62.114 bolsas: el 99,48 %. Los errores de dos grupos o más son 321 casos, el 0,52 %. Confundir Bajo con Alto —el peor error posible— ocurre una vez en 13.323 bolsas Bajas.
- **Ningún grupo queda sin predicciones.** Las cuatro columnas se usan. Es un contraste deliberado con la línea base de clase mayoritaria, que asigna todo a Medio y abandona los otros tres grupos por completo: alcanza un 43,8 % de acierto sin distinguir absolutamente nada, y por eso el acierto bruto no es la métrica que se reporta.
- **El error dominante es de un solo escalón, hacia abajo.** Las confusiones más numerosas son Bajo leído como Medio (2.308) y Medio-Alto leído como Medio (2.232). En una lista de priorización esto se traduce en un reordenamiento local entre vecinos, no en dejar fuera un vano crítico.

Para un uso de priorización, ese perfil es el deseable: importa más no perder los vanos Altos que afinar la frontera entre Bajo y Medio, y es precisamente en los Altos donde el modelo es más preciso.

7.8.4. Un caso completo: de la inferencia al diagnóstico y a la simulación

Las tres evidencias anteriores describen al modelo en conjunto. Esta última recorre un caso concreto de principio a fin, que es la forma en la que el sistema se usa realmente. Todos los valores provienen de una ejecución archivada del circuito **HER23L16**, con su fecha y su identificador de corrida, y son reproducibles desde ella.

Punto de partida: el circuito. HER23L16 ocupa la **posición 2 entre los 208 circuitos** del ranking y pertenece a la banda de Riesgo Alto. En el período de análisis registró 130 eventos y 507.228 unidades de UITI, frente a promedios de red de 31 eventos y 84.890 unidades: alrededor de seis veces la carga media. Es, por tanto, un circuito donde el sistema debe tener algo útil que decir.

Paso 1 — Inferencia: puntuar cada celda vano × ventana. Se elige la ventana **V8**, del 15 de febrero al 14 de marzo de 2026. En esa ventana el circuito tiene **197 vanos con registros**, es decir 197 bolsas. El modelo estima el UITI de cada una y la regla de asignación

le pone su grupo. La mediana del UITI estimado en la ventana es de 84,05 unidades y la celda más cargada llega a 2.882,32.

Paso 2 — Diagnóstico: qué variable pesa, y en cuántos vanos se puede mover.

Para cada vano, el mecanismo de atención indica qué variables sostuvieron esa estimación. El diagnóstico agrega esa lectura sobre los 197 vanos y separa lo que *se puede intervenir* de lo que *solo se puede observar*. Esa separación es deliberada: el viento y la lluvia explican, pero no se contratan.

Tabla 17: Diagnóstico de la ventana V8 de HER23L16: variables ordenadas por su alcance sobre los 197 vanos. “Baja de grupo” cuenta en cuántos vanos mover esa sola variable hasta su mejor valor bastaría para que el vano descienda de grupo de criticidad.

Variable	Naturaleza	Vanos donde pesa	Baja de grupo	Mejor valor típico
CONDUCTOR	Intervención	194	4	4/0-CU-AISLADO
VAL_CRIT_APOYO	Intervención	166	1	10,0
NR_T (vegetación)	Intervención	164	6	116,0
TIPO	Intervención	106	3	1CC
Ráfaga de viento	Escenario	167	38	—
Viento	Escenario	159	23	—
Temperatura	Escenario	160	18	—

La tabla se lee así: el conductor pesa en casi todos los vanos de la ventana, pero cambiarlo solo alcanzaría para bajar de grupo a cuatro de ellos. La vegetación pesa en 164 y alcanzaría en seis. Y las variables de escenario — viento, ráfaga, temperatura — son las que más veces bastarían por sí solas (38, 23 y 18 vanos), lo que es a la vez informativo y una advertencia: **gran parte de la criticidad de esta ventana la explica el clima, que no se interviene**. Presentarlas juntas sin distinguir su naturaleza produciría un plan de trabajo imposible de ejecutar; por eso el sistema las separa antes de recomendar nada.

Paso 3 — Simulación: qué pasaría si se interviene. El simulador toma los quince vanos más cargados de la ventana, restringe los controles a las once variables *intervenibles*, y busca de forma incremental la combinación que más reduce el UITI estimado. La Tabla 18 muestra el plan completo para el vano más crítico.

Tabla 18: Plan de intervención simulado para el vano 20607133, el más cargado de la ventana V8. Cada fila añade un cambio al anterior; la última columna es el UITI estimado tras aplicarlos de forma acumulada.

Paso	Variable	Valor propuesto	UITI estimado	Reducción acumulada
—	<i>estado actual</i>	—	2.882,32	—
1	VAL_CRIT_APOYO	10,0	2.618,52	9,2 %
2	NR_T (vegetación)	0,0	2.581,19	10,4 %
3	NG_RED	1,0	2.546,86	11,6 %
4	ALTURA	18,0	2.521,07	12,5 %

Paso 4 — La lectura honesta del resultado. El plan reduce el UITI estimado de ese vano un 12,5 %, pero **no lo hace descender de grupo**: entra Alto y sigue Alto. Lo mismo ocurre en los quince vanos simulados: la reducción mediana es del 12,5 % en V8 y del 12,3 %

en V9 —con un máximo del 22,6 %—, y **ninguno cambia de grupo**. En la ventana V11 las reducciones son mayores, con mediana del 32,5 % y máximo del 45,9 %, y el resultado es el mismo.

Ese resultado no es un fallo del modelo ni del simulador: es la respuesta correcta, y es justamente lo que se le pide a una herramienta de decisión. Dice tres cosas útiles y una incómoda pero necesaria:

1. **Cuánto se gana, y con qué.** Un 12,5 % de reducción sobre el vano más cargado del circuito peor rankeado es una mejora real y cuantificada, con las cuatro acciones concretas que la producen.
2. **En qué orden conviene actuar.** El primer paso —la criticidad del apoyo— aporta 9,2 de los 12,5 puntos. Los tres siguientes suman entre todos 3,3. Si el presupuesto alcanza para una sola intervención, la tabla ya dijo cuál.
3. **Cuánto cuesta.** Cada uno de esos pasos se cotiza contra el catálogo de actividades del contrato de CHEC (Sección 8), de modo que la decisión se toma comparando reducción esperada contra costo, y no sobre una intuición.
4. **Lo incómodo: intervenir la infraestructura no basta en este vano.** Con las once variables intervenibles llevadas a su mejor valor, el vano sigue en el grupo Alto. La razón está en el diagnóstico del paso 2: en esta ventana la mayor parte de la criticidad la aportan variables de escenario —viento y ráfaga— que ninguna orden de trabajo modifica. El sistema lo dice explícitamente en lugar de ofrecer un descenso de grupo que no ocurriría en campo.

Ese último punto es la diferencia entre una herramienta que informa una decisión y una que promete resultados. El simulador mantiene además desacoplados el efecto simulado y el costo cotizado: no multiplica uno por otro para producir una cifra de beneficio, porque esa cifra tendría apariencia de estimación sin respaldo que la sostenga.

7.9. Dónde se entrena

El cuaderno resuelve el dispositivo de cómputo automáticamente, y esa elección resultó equivocada al medirla. Sobre un pliegue real de 88.987 bolsas se cronometraron tres épocas y se proyectó la validación cruzada completa.

Tabla 19: Tiempo y memoria del entrenamiento por dispositivo, medidos en el equipo de referencia

Dispositivo	s / época	Proyección completa	Pico de memoria
CPU, 2 hilos	3,28	8,2 min	3.293 MB
CPU, 4 hilos	3,61	9,0 min	3.288 MB
CPU, 8 hilos	4,60	11,5 min	3.275 MB
CPU, 14 hilos	5,51	13,8 min	3.267 MB
GPU integrada (MPS)	19,63	49,1 min	11.234 MB

Los resultados evidencian dos comportamientos no anticipados. La GPU integrada del propio procesador resulta seis veces más lenta y demanda 3,4 veces más memoria que la CPU del mismo equipo. Y un mayor número de núcleos empeora el tiempo de forma monótona, porque coordinar hilos cuesta más que el trabajo que reparten.

Por qué la GPU resulta más lenta, medido. La explicación intuitiva —que el modelo, con 150.926 parámetros, sería demasiado pequeño para amortizar el traslado de cada lote— se

verificó y **no se sostiene**. El desglose de un paso de entrenamiento muestra que el traslado de datos es despreciable: 0,80 ms de transferencia y 0,23 ms de armado del lote, frente a un paso completo de 172 ms. Y una red equivalente en tamaño, entrenada sobre lotes del mismo número de instancias, corre *más rápido* en la GPU integrada que en la CPU (0,90 ms contra 1,21 ms), de modo que el tamaño del modelo tampoco es el factor.

La causa real está en la estructura de la bolsa. Las bolsas no tienen todas el mismo número de instancias, así que cada lote de 256 bolsas presenta un total distinto de filas: entre 560 y 793 en la medición, con 34 valores diferentes en 40 lotes consecutivos. El motor de cómputo de la GPU integrada compila y almacena sus núcleos de cálculo **indexados por la forma exacta del tensor**, de manera que cada forma nueva obliga a recompilar. La CPU es indiferente a ese cambio.

Tabla 20: Costo de un paso de entrenamiento según la forma del lote, sobre el mismo modelo y el mismo código. La única diferencia entre las dos filas es si el lote cambia de tamaño entre iteraciones.

Régimen	CPU	GPU integrada	Sobrecosto
El mismo lote repetido (forma fija)	9,99 ms	15,12 ms	1,5×
Lotes distintos (forma variable, el caso real)	9,03 ms	106,27 ms	11,8×

Siete de esas once veces provienen únicamente de la recompilación por cambio de forma. La memoria confirma el mecanismo: con forma fija el controlador reserva 62,9 MB y con forma variable 129,6 MB, mientras los tensores efectivamente vivos son los mismos en ambos casos —alrededor de 24 MB—. El exceso es caché de núcleos compilados, no datos.

Dos consecuencias, y conviene no confundirlas:

- **No hay un defecto de formulación.** El cálculo del modelo es idéntico en los dos dispositivos y produce los mismos resultados. Lo que aparece es una interacción entre una propiedad legítima del problema —las bolsas tienen cardinalidad variable— y la forma en que ese motor de cómputo almacena sus núcleos.
- **La interacción es mitigable, pero no cambia la decisión.** Rellenar los lotes hasta un número fijo de instancias recuperaría ese factor de siete. Aun así la GPU integrada quedaría un 50 % por encima de la CPU, de modo que la conclusión se mantiene: no hay motivo para trasladar este entrenamiento a la GPU, y la optimización solo tendría sentido si en el futuro se entrenara sobre un volumen que la CPU ya no absorbiera.

Ninguna parte del proyecto requiere GPU. Un equipo de dos o cuatro núcleos y 8 GB de memoria reentrena este modelo en unos diez minutos. El pico de memoria lo fija la preparación de datos — leer el archivo de eventos cuesta 2,4 GB — y no el entrenamiento.

Entonces, ¿hace falta o no una estación de trabajo? La afirmación anterior tiene un alcance preciso y conviene no extenderlo: **lo que se descarta es la GPU**, no el equipo. El proyecto no necesita una tarjeta gráfica dedicada en ninguna de sus partes, y por lo tanto no justifica la compra de una estación de trabajo *por motivo de cómputo acelerado*.

El equipo sí importa, pero por otras razones y en otra escala. La Tabla 21 resume lo medido.

Tabla 21: Qué exige cada parte del sistema al equipo de trabajo, según lo medido

Uso	RAM	Núcleos	Lo que manda el costo
Solo los cuatro tableros	4 GB	2	Nada: son HTML ya construido; el navegador solo filtra.
Además el simulador	8 GB	4	El modelo corre en CPU sobre lo que el usuario escriba.
Además reentrenar el modelo	8 GB	2	La <i>preparación</i> de datos, no el entrenamiento.
Además generar informes	8 GB	4	La conexión y la cuota de suscripción, no el equipo.
Todo, con holgura	16 GB	4–8	Trabajar con varias partes abiertas a la vez.

Tres conclusiones se desprenden de esa tabla:

- **Un equipo de escritorio corriente basta.** Cuatro núcleos, 16 GB de memoria y 20 GB de disco libre cubren la operación completa con holgura. Ese es el requisito real, y está por debajo de lo que suele entenderse por estación de trabajo.
- **Lo que exige memoria es leer los datos, no entrenar.** El pico de 2,4 GB aparece al abrir el archivo de registros; el entrenamiento propiamente dicho se sostiene por debajo de eso. Aumentar núcleos no ayuda y, medido, empeora el tiempo.
- **La generación de informes no pide máquina, pide conexión y cuota.** El razonamiento en lenguaje natural ocurre fuera del equipo. Mientras los agentes trabajan, la máquina local está prácticamente ociosa.

En síntesis: no se requiere una estación de trabajo de altas prestaciones ni GPU, pero sí un equipo con 16 GB de memoria si se pretende usar varias partes del sistema simultáneamente. Para cargas mayores — reprocesar la totalidad de la historia o entrenar variantes en paralelo — la ruta prevista es Azure Databricks, descrita en la Sección 10, y no un equipo local más grande.

8. Simulador contrafactual “¿Qué pasa si...?”

El simulador es el único tablero que mantiene un proceso de Python en ejecución, porque cada simulación exige una inferencia nueva del modelo. Su flujo consiste en seleccionar un circuito y una ventana, marcar vanos, generar un diagnóstico de intervención, modificar las variables elegidas y comparar la criticidad observada con la simulada junto con el costo asociado.

El simulador responde una pregunta distinta de la que responde la atribución de importancia. La atribución indica qué explica el `UITI_VANO` observado; el simulador indica qué variable modificar y hasta qué valor para que el vano descienda de grupo. Las respuestas no coinciden: medido sobre vanos reales, el ranking por sensibilidad y el ranking por reducción alcanzable no comparten ninguna de sus cinco primeras variables, lo que justifica calcular el segundo en lugar de derivarlo del primero.

8.1. Flujo de una simulación

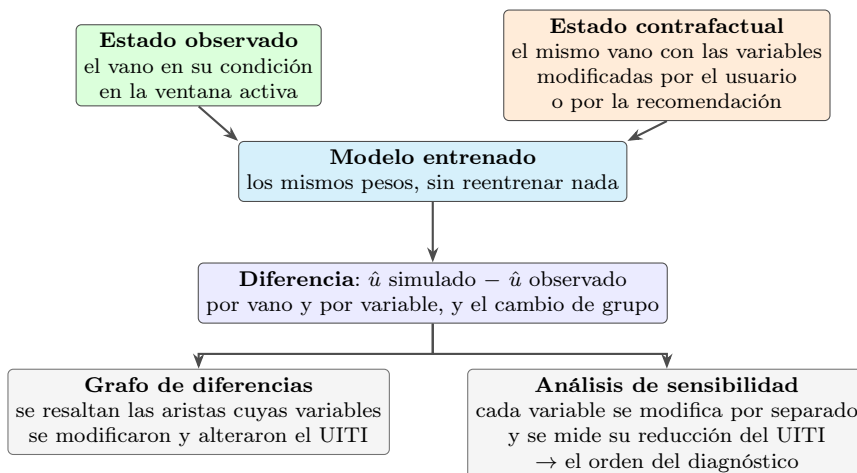


Figura 13: Uso del modelo entrenado en la simulación. El mismo artefacto se evalúa dos veces sobre la misma ventana; la diferencia entre ambas salidas produce el grafo de diferencias y el análisis de sensibilidad. Las aristas que no producen cambio no se representan.

Las decisiones de interacción que gobiernan el tablero son las siguientes:

- **Selección de vanos por tres mecanismos acumulativos.** Un botón por grupo de criticidad marca los vanos de ese grupo en la ventana fijada; también es posible la selección individual mediante casillas o directamente sobre el mapa. La selección de un grupo tras otro es acumulativa, y la acción de desmarcar es la única que elimina la selección vigente. El panel informa en todo momento cuántos vanos corresponden al grupo y a la ventana seleccionados, con su fecha.
- **El diagnóstico se calcula solo sobre lo marcado.** La selección acota la consulta, no la determina. Si no existe selección explícita, se utilizan los quince vanos con mayor UITI de la ventana activa. Cuando existen vanos con eventos por fuera de la selección, el tablero informa esa condición y permite incorporarlos.
- **Dos listas separadas.** El diagnóstico produce dos listas ordenadas por reducción media del UITI — variables de intervención y variables de escenario —, cada una con su valor sugerido y su reducción estimada, e identifica cuándo la modificación de una sola variable basta para cambiar de grupo a por lo menos un vano.
- **Los valores pueden ajustarse manualmente.** Cada vano seleccionado dispone de un panel con sus variables, inicializadas con el valor observado en la ventana activa — mediana para las numéricas y moda para las categóricas —, nunca con un valor genérico.
- **Los dos mapas comparten circuito, ventana, escala y topología.** La diferencia entre ambos está únicamente en el color de criticidad. El encuadre difiere de forma deliberada: el mapa izquierdo conserva la vista del circuito completo como referencia y el derecho se aproxima a los vanos seleccionados; cada uno dispone de su propio control de centrado.

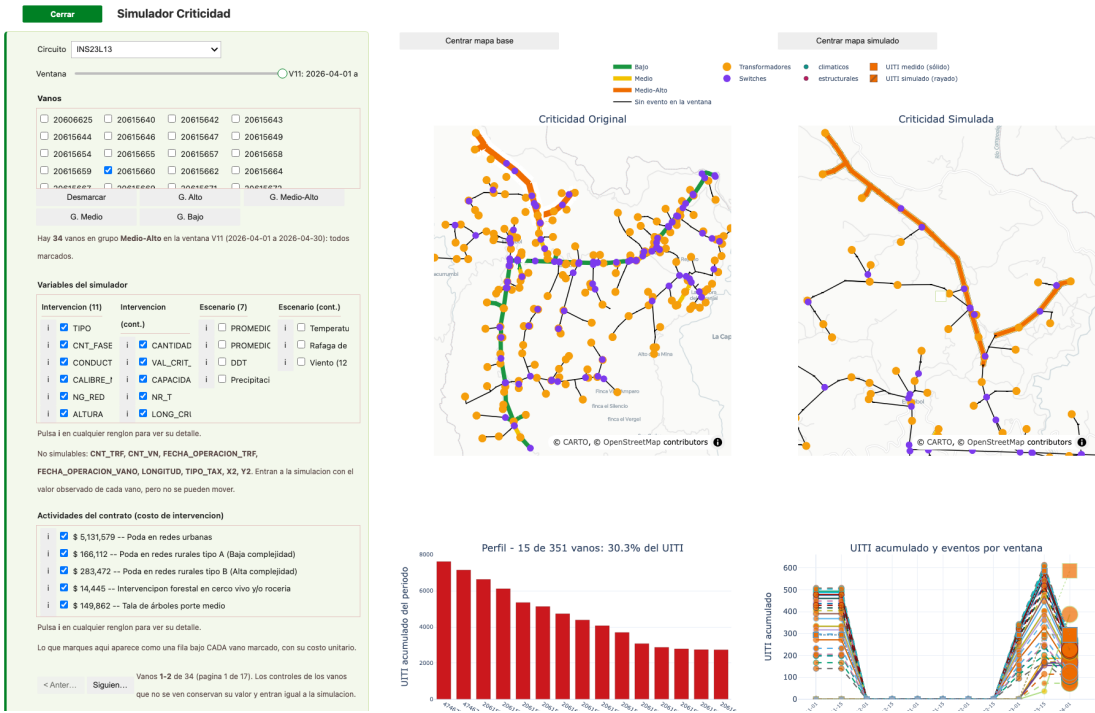


Figura 14: Simulación contrafactual del circuito INS23L13 en la ventana V11, del 1 al 30 de abril de 2026. Se presentan el panel de control, los mapas de criticidad original y simulada, el perfil del circuito y la serie de UITI acumulado y eventos por ventana.

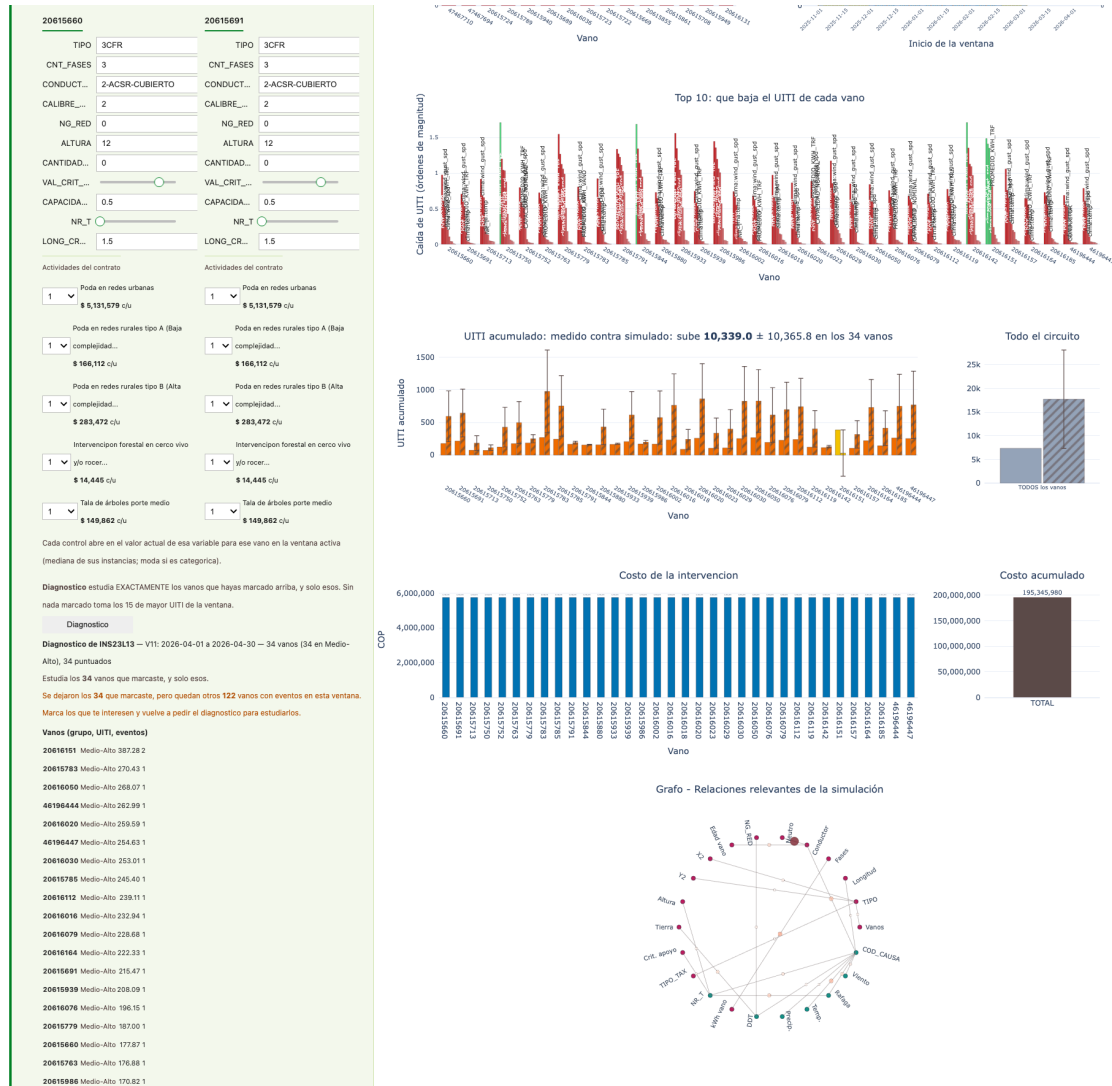


Figura 15: Resultados de la misma ejecución: la reducción del UITI que aporta cada variable en cada vano, el UITI observado frente al simulado, el costo por vano y el costo acumulado — 195.345.980 COP sobre cinco actividades del contrato — y el grafo de relaciones relevantes. El escenario simulado no produce mejora: el UITI acumulado sube 10.339,0 con un desfase del modelo de $\pm 10.365,8$, y el propio título del panel lo declara. Se trata de un resultado legítimo y no de un error de ejecución: el margen es del orden del cambio, de modo que esta corrida no sostiene una diferencia en un sentido ni en el otro. Publicar la cifra sin ese margen convertiría el sesgo del modelo en un ahorro.

8.2. Clases de variable y criterios de separación

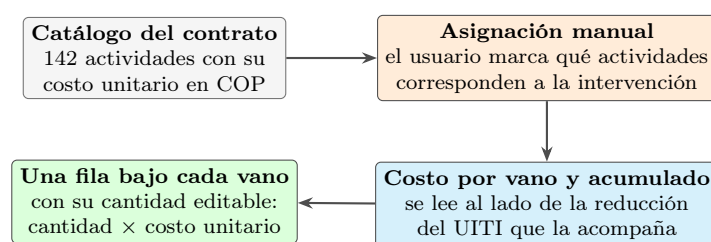
No todas las variables del modelo son modificables, y las que sí lo son no tienen el mismo significado. El simulador las separa en tres clases, y la separación tiene implicaciones funcionales en la interpretación del escenario: determina qué pregunta se está respondiendo. Esta capa evita que el panel presente como equivalentes intervenciones de naturaleza distinta, por ejemplo el desplazamiento de las coordenadas geográficas de un vano y la reducción de su riesgo por vegetación.

Tabla 22: Clases de variable del simulador y criterio que las separa

Clase	Variables	Criterio
Intervención 11 variables	TIPO, CNT_FASES, CONDUCTOR, CALIBRE_NEUTRO, NG_RED, ALTURA, CANTIDAD_TIERRA, VAL_CRIT_APOYO, CAPACIDAD_NOMINAL, NR_T, LONG_CRUCETA	Atributos del activo que CHEC puede modificar mediante una obra. Cada una corresponde a una actividad ejecutable del contrato — cambio de conductor, elevación del apoyo, mejora de la puesta a tierra o poda —. Una variable sin obra asociada que la modifique no se considera de intervención.
Escenario 7 variables	DDT, precipitación, temperatura, viento y ráfaga (con 12 rezagos cada una), PROMEDIO_KWH_TRF, PROMEDIO_KWH_VANO	Condiciones del entorno y de la demanda que no son controlables mediante una intervención operativa, pero que sí pueden suponerse. Combinarlas con las de intervención podría atribuir a una obra un efecto derivado en realidad de condiciones ambientales favorables, de modo que se simulan por separado y el tablero indica cuál de los dos componentes se aplicó.
No modificables 8 variables	CNT_TRF, CNT_VN, FECHA_OPERACION_TRF, FECHA_OPERACION_VANO, LONGITUD, TIPO_TAX, X2, Y2	Ingresa a la simulación con el valor observado de cada vano, porque el modelo las requiere, pero no admiten modificación: son atributos de identidad, contexto o topología. Modificarlas no simula una intervención sino una entidad distinta, y reduciría la validez interpretativa del resultado.

8.3. Incorporación de los costos por actividad de contrato

La valoración económica se apoya en un único insumo, el libro de precios del contrato de CHEC — `Actividades_mantenimiento_costos_2026.xlsx` —, que aporta 142 actividades con su costo unitario en pesos colombianos. Ese archivo se lee mediante un módulo específico y no con una lectura directa de hoja de cálculo, porque es la exportación de una tabla dinámica y requiere tres correcciones deterministas: descartar la fila de total general, que ofrecida como actividad introduciría el promedio de todo el catálogo; apartar y enumerar de forma explícita las doce actividades que llegan sin costo unitario, en lugar de omitirlas en silencio; y reparar la codificación de los nombres, dado que el catálogo carece de código de ítem y el nombre opera como clave.

**Figura 16:** Incorporación de los costos por actividad. El modelo no deduce qué actividad corresponde a cada modificación: la asignación es del usuario.

El propósito del simulador no es aproximar por cercanía textual, sino cotizar el plan que el usuario declara. La rejilla del panel recibe las actividades marcadas por vano con sus repeticiones y devuelve el total de cada vano junto con su desglose ordenado de mayor a menor subtotal. Un vano sin actividades conserva su fila con valor cero, porque ese cero indica que la simulación movió su riesgo sin obra asociada. Una actividad ausente del catálogo interrumpe el cálculo en lugar de valorarse en cero, dado que un presupuesto incompleto no sería distinguible en pantalla de uno correcto.

Las dos mitades permanecen deliberadamente desacopladas: seleccionar una actividad de poda no modifica la variable de riesgo por vegetación, y reducir esa variable no programa una

poda. El modelo no dispone de una correspondencia entre actividades contractuales y variables, y establecerla sin respaldo produciría una cifra con apariencia de estimación de beneficio que no lo sería. Lo que el sistema presenta lado a lado es el efecto simulado y el costo cotizado del plan declarado; la unión de ambos permanece como decisión del usuario hasta que CHEC provea esa correspondencia.

Los importes que el sistema despliega proceden en su totalidad del libro de precios vigente y se recalculan en cada ejecución. El presente informe no los reproduce, dado que la cuantificación final depende de cantidades, alcance de obra, zona, disponibilidad operativa y validación contractual de CHEC.

8.4. Archivo y recuperación de una simulación

Bajo los controles de simular y limpiar, el panel dispone de las acciones de guardar y cargar. La acción de guardar permanece deshabilitada mientras no exista una simulación en pantalla, dado que archivar un tablero vacío no constituye un registro.



Figura 17: Bloque de archivo del simulador. La ruta de la carpeta se publica de forma permanente — desde la apertura del tablero y con independencia de que se haya guardado — en el renglón situado bajo la lista desplegable. Tras guardar se indican los nombres de los dos archivos escritos.

Cada ejecución archivada produce dos archivos con el mismo nombre y distinta extensión.

Tabla 23: Archivos que produce la acción de guardar, con tamaños medidos sobre la ejecución de las Figuras 14 y 15

Archivo	Tamaño	Contenido
<circuito>_<ventana>_-<fecha>.html	7,1 MB	El informe: los ocho paneles de la simulación, más tres tablas — los vanos con las variables simuladas y su valor fijado; las actividades de contrato por vano con número de intervenciones, costo unitario, descripción y costo total; y el UITI medido frente al simulado por vano, con su porcentaje de variación y los totales de ambas columnas —. Es autocontenido: incorpora la biblioteca de graficación, por lo que se abre sin conexión y sin requerir instalación.
<circuito>_<ventana>_-<fecha>.simchec.json.gz	6,9 KB	El registro con el que la acción de cargar reconstruye la simulación: circuito, ventana, los vanos seleccionados, las variables con su valor y las actividades con sus cantidades.

El registro no almacena las figuras, y esa es la razón de la diferencia de tres órdenes de magnitud entre ambos archivos. La totalidad de lo que se presenta en pantalla se deriva de

ejecutar el modelo sobre las entradas, de modo que almacenarlo equivaldría a guardar el valor de retorno de una función junto a sus argumentos: dos representaciones de lo mismo que divergen en el momento en que se reentrena el modelo. El registro almacena las entradas, y la acción de cargar las repone y vuelve a ejecutar el modelo.

El costo de esa decisión — que un modelo reentrenado devuelva valores distintos — se asume de forma explícita: el registro incorpora la firma criptográfica de los artefactos con los que se ejecutó, y al cargar el panel advierte cuando no coinciden. El informe HTML, por su parte, conserva de forma íntegra lo que se decidió ese día. El formato es gzip de JSON y no un formato propietario, de modo que una auditoría posterior puede abrirlo sin este programa.

Tabla 24: Destino de las simulaciones archivadas según el entorno de ejecución

Entorno	Dónde se almacena	Cómo se recupera
Local, macOS y Windows	~/CriticidadCHEC/simulaciones, o bien C:\Users\<usuario>\-CriticidadCHEC\simulaciones	El informe se abre con doble clic. La carpeta depende del directorio personal del usuario y no de la aplicación: esta se reconstruye automáticamente cuando cambian los datos y en Windows puede trasladarse a una ruta más corta, de modo que el trabajo del usuario no debe residir en ella.
Databricks	/Volumes/<catálogo>/<esquema>/chec-simulador/simulaciones	Se descarga desde la sección de volúmenes del catálogo, en la interfaz del espacio de trabajo. La aplicación no puede ofrecer la descarga directamente, porque el servidor que la publica no sirve archivos individuales.

El almacenamiento en el Volume y no en el contenedor es una condición necesaria: el disco del contenedor es efímero — se elimina en el siguiente despliegue — y el usuario no dispone de acceso a él. La selección entre ambos destinos la determina una variable de entorno que el comando de despliegue resuelve y registra en la configuración de la aplicación; no se infiere del entorno de ejecución, dado que una inferencia de ese tipo produciría un tablero local intentando escribir en un Volume inexistente. La carpeta se crea durante el despliegue y es hermana de la del paquete precalculado, nunca interna a ella: el paquete se elimina y se vuelve a crear en cada despliegue, por lo que ubicar las simulaciones en su interior eliminaría el trabajo almacenado en cada actualización. La aplicación requiere el privilegio de escritura sobre ese Volume; sin él, el tablero opera con normalidad y únicamente falla la acción de guardar.

8.5. Rendimiento medido

La implementación no establece un límite explícito de vanos seleccionados; el rendimiento depende de los recursos disponibles, y el diagnóstico, la aplicación de recomendaciones y los informes cubren todos los vanos seleccionados. La prueba de mayor tamaño registrada corresponde al circuito DON23L13 con 407 vanos: 3,9 s de selección, 2,7 s de diagnóstico y 6,5 s de aplicación. El plan fija 685 valores sobre 9 variables distintas, y la retícula queda con 3.663 controles.

9. Agentes e informes automáticos

El simulador proporciona la exploración interactiva; los agentes generan la documentación técnica de los resultados. Ambos componentes utilizan el mismo motor analítico: el diagnóstico que el simulador muestra en pantalla — la reducción del UITI que aporta cada variable en cada vano — es exactamente el que recibe y redacta el rol **inference**. Los agentes complementan la salida del tablero con contexto histórico, análisis de informes técnicos en PDF y comparación de evidencia.

9.1. Qué es una habilidad y cómo se ejecuta

Una *habilidad* es un contrato declarativo en formato Markdown, versionado en el repositorio bajo `.claude/skills/`, que especifica a un agente su tarea, sus insumos, sus restricciones y la forma exacta del resultado esperado. No constituye código ejecutable: es la especificación que el entorno de agente carga previamente al razonamiento.

Esa separación es la que hace verificable el sistema. El cálculo lo realiza Python de forma determinista y la habilidad gobierna únicamente la interpretación, sujeta a tres reglas transversales:

- **Restricción al contexto entregado.** La habilidad solo puede citar fechas, variables, escenarios y puntos críticos contenidos en el paquete de contexto; una referencia ausente de ese paquete no puede figurar en su salida.
- **Forma de salida fijada.** Cada habilidad de razonamiento declara un contrato JSON con secciones obligatorias, que se valida antes de integrarse al informe. Si tras los reintentos de reparación no existe un objeto válido, el flujo detiene la generación del informe en lugar de publicar una salida que incumpla el contrato.
- **Invocación por referencia.** Una habilidad que requiere a otra la invoca en lugar de replicar su lógica, de modo que una modificación del motor de reporte se propaga a todos sus consumidores sin sincronización manual.

9.2. Informe por circuito

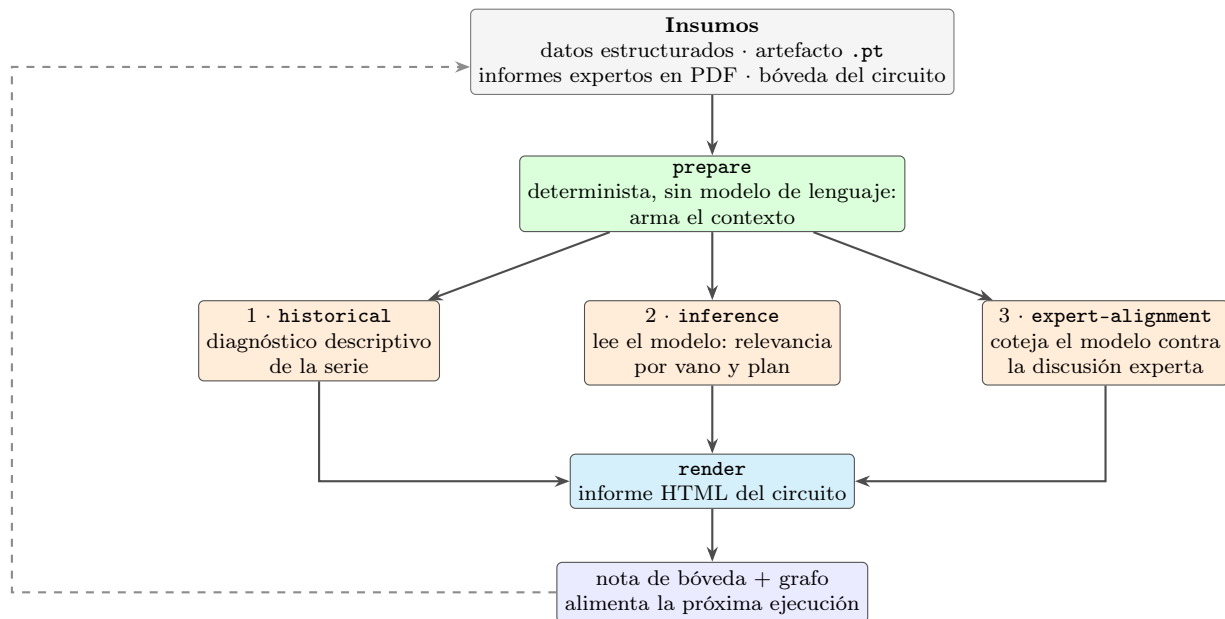


Figura 18: Flujo del informe por circuito. La preparación y el renderizado son deterministas y se ejecutan sin modelo de lenguaje; únicamente los tres roles intermedios lo utilizan. La nota de bóveda que produce cada corrida alimenta la siguiente.

Tabla 25: Roles de agente: insumo, función y producto

Rol	Insumo	Producto
historical	Selección de circuito y ventana, serie diaria, puntos críticos, causas observadas, variables disponibles y faltantes, y relaciones del grafo.	Objeto JSON con el diagnóstico descriptivo del comportamiento de UITI_VANO : secciones de análisis, referencias a fechas y puntos críticos existentes, hipótesis históricas y limitaciones declaradas.
inference	Relevancia por vano y por variable derivada del modelo, escenarios por ventana, puntajes por modo y metadatos del artefacto.	Objeto JSON con la interpretación predictiva por escenario. No puede priorizar UITI_VANO como predictor ni citar escenarios que el flujo no generó, y debe declarar que las relevancias y los grafos explican asociaciones, no causalidad.
expert-alignment	Los dos objetos JSON validados anteriores, las filas expertas ya filtradas por cercanía temporal y la lista exacta de variables del modelo.	Objeto JSON con coincidencias, diferencias, brechas, hasta cinco variables priorizadas para revisión operativa y una síntesis. Solo puede citar evidencia de las filas comparadas.
pdf-discussion-extraction	Un fragmento de informe técnico con sus metadatos: archivo de origen, páginas, circuito heredado del nombre del archivo, periodo y ventana solicitada.	Decisión de incluir o descartar. Si incluye, una fila con circuito, fecha o intervalo normalizado, análisis breve y evidencia textual del propio fragmento.

9.3. Informe gerencial por banda de riesgo

La síntesis transversal parte de una banda de riesgo de la clasificación descrita en la Sección 6.5. Si la banda contiene más de doce circuitos se toman los doce más críticos, y el informe declara de forma explícita que **describe la cola de la banda y no su circuito típico**. El panorama que sintetiza tiene por universo el grupo completo, nunca la muestra: las barras del conjunto abarcan los 208 circuitos con la muestra resaltada, y el grafo radial de causas reúne los patrones que cruzan circuitos.

Tabla 26: Comandos de informe y de datos, con su alcance

Comando	Salida generada	Alcance
/report	Informe HTML de un circuito, con figuras y trazabilidad por afirmación.	Un circuito, una ventana.
/reporte-lote	Encadena /report sobre una lista de circuitos.	Varios circuitos.
/informe-gerencial	Síntesis por banda de riesgo, con barras del conjunto completo de circuitos y grafo radial de causas.	Hasta 12 circuitos de una banda.
/clima	Enriquece los datos con Open-Meteo, bajo tres validaciones de cuota.	Toda la tabla elegida.
/vault-circuito	Proyecta el circuito a una nota Markdown e indexa el grafo de conocimiento.	Acumulativo entre ejecuciones.

Costo por informe. Un informe por circuito consume del orden de 318.760 unidades de texto del modelo de lenguaje. El requisito determinante no es la capacidad de cómputo local sino **la cuota del modelo**: mientras los tres roles razonan, el consumo de recursos locales de cómputo

permanece limitado durante esta etapa.

10. Migración a Azure Databricks

El despliegue en Databricks es independiente y no mantiene sincronización en tiempo real con el entorno local: allí se ejecuta la misma cadena, con sus propios datos cargados y sus propias aplicaciones desplegadas. La migración se ejecuta mediante un único comando, `/subir-a-databricks`, que transfiere tres grupos de artefactos.

10.1. La interfaz de línea de comandos como estrategia central

El uso de la interfaz de línea de comandos de Databricks no constituye un detalle de implementación: es el mecanismo que permite a un agente de código conducir el despliegue completo desde la terminal, sin recurrir a la interfaz web del espacio de trabajo. El comando se comunica con Databricks exclusivamente por esa vía — 51 invocaciones —: la apertura de sesión por OAuth, el descubrimiento del catálogo y del Volume, la copia de archivos, la importación del cuaderno y la creación y despliegue de las aplicaciones. De ahí que el usuario solo aporte la URL del espacio de trabajo: el perfil se deriva de esa URL y todo lo demás se resuelve en tiempo de ejecución.

Tabla 27: Artefactos que transfiere el despliegue

#	Artefacto	Destino	Contenido
1	<code>data/</code>	Volume de Unity Catalog	Aproximadamente 945 MB: el archivo de eventos (566), la caché de bolsas (199), los tres archivos geográficos con sus complementos (180), el modelo entrenado, el grafo y los catálogos en hoja de cálculo.
2	Dos aplicaciones	Databricks Apps, cada una en su recurso de cómputo	La aplicación de criticidad sirve los cuatro tableros en cuatro rutas; el simulador requiere un intérprete de Python en ejecución.
3	<code>notebooks/05_mil_vano_ventana.ipynb</code>	Cuaderno del espacio de trabajo	El registro legible del modelo, importado sin salidas por el tope de 10 MB del importador.

10.2. Secuencia de ejecución

El comando solicita un solo dato al usuario — la URL del espacio de trabajo de destino — y todo lo demás se resuelve o se descubre en tiempo de ejecución.

Tabla 28: Las ocho etapas de /subir-a-databricks

#	Etapas	Comportamiento
1	Solicitud de la URL del espacio de trabajo	Es el único dato que se solicita, y se solicita en cada ejecución: nunca se deduce de un perfil con sesión abierta, dado que un destino incorrecto no produce ningún error observable.
2	Resolución del perfil y de la identidad	Se derivan de esa URL, una sola vez, y la identidad se confirma con una llamada real antes de transferir datos.
3	Resolución del destino en Unity Catalog	El catálogo se descubre en tiempo de ejecución; nunca se asume.
4	Validación de los datos en el Volume	Se verifican primero los archivos existentes y sus tamaños. Solo se transfieren los faltantes: retransmitir los 566 MB ya presentes es la operación más costosa del proceso.
5	Validación de las aplicaciones	Una aplicación se considera operativa cuando su estado es activo, su proceso está en ejecución y su último despliegue fue satisfactorio, de forma simultánea. Una aplicación operativa y actualizada no requiere un nuevo despliegue.
6	Validación del cuaderno	Se importa sin salidas ni contadores de ejecución. Si la etapa de datos quedó bloqueada, el cuaderno queda disponible pero no podrá ejecutarse: esa condición se registra como <i>degradado</i> , no como satisfactoria.
7	Registro y publicación de la bitácora	En una rama, nunca sobre la rama principal.
8	Generación del reporte de ejecución	Una fila por etapa, la ruta de la bitácora y cada restricción junto con el responsable de su resolución.

10.3. Trazabilidad del despliegue

La etapa final produce dos tablas cuyo propósito es que una corrida quede auditable sin releer la salida de la terminal.

La primera resume el proceso con una fila por etapa — siempre las tres etapas de transferencia, incluso las que no hicieron falta o quedaron bloqueadas —, con columnas de lo que ya estaba, lo que se subió, el estado y **la causa**. La columna de causa la llenan únicamente las etapas que no terminaron de forma satisfactoria, e indica *por qué*, no *qué*. Una etapa que no finalizó satisfactoriamente y no declara causa se reporta de forma explícita como carente de causa registrada: una celda vacía junto a un fallo se leería como un fallo sin motivo, y eso constituiría otra afirmación.

La segunda es la tabla de ubicaciones, que responde la pregunta que se formula cuando el despliegue finaliza satisfactoriamente: la ubicación de cada artefacto desplegado. Reúne el Volume de datos, el paquete precalculado del simulador, la carpeta de simulaciones guardadas, las dos aplicaciones con su URL literal y el cuaderno del espacio de trabajo, cada uno con su ruta y su camino de acceso en la interfaz. Esas rutas no son deducibles, porque el catálogo y el esquema se resuelven en cada corrida.

La bitácora se escribe en formato Markdown bajo **reports/despliegues/** y sanea de su contenido cualquier credencial — tokens de acceso, secretos de cliente y contraseñas — antes de registrarla. El despliegue no modifica ningún cuaderno del repositorio y no integra nada en la rama principal.

11. Instalación y puesta en marcha

La puesta en marcha se compone de seis pasos, de los cuales los dos últimos aplican únicamente si el proyecto se despliega en Databricks. Los pasos 3, 4 y 6 se ejecutan desde un agente de código y no requieren instalar dependencias manualmente.

11.1. Los seis pasos

1. **Clonar el repositorio**, con `git lfs` instalado *antes* de clonar: `git lfs install`, luego `git clone` y, dentro de la carpeta, `git lfs pull`. Este último paso no es opcional: sin él, la base de eventos de 566 MB y la caché de bolsas de 199 MB se descargan como punteros de 134 bytes, que existen en el sistema de archivos y no contienen datos, de modo que el fallo se manifiesta mucho después y sin referencia a esta causa. En Windows, la ruta del clon debe caber en 61 caracteres; el repositorio incluye un guion que lo resuelve sin requerir permisos administrativos.
2. **Abrir el agente de código**. Todo lo que sigue se ejecuta desde un agente. El repositorio publica sus quince comandos para tres entornos — Claude Code, que es la fuente del contrato, OpenCode y VS Code Copilot — y la invocación es idéntica en los tres. Se abre el agente de preferencia en la carpeta del clon.
3. **Instalar dependencias con `/instalar-local`**. El comando ejecuta primero un diagnóstico con el intérprete de Python del sistema y solo la biblioteca estándar, dado que debe funcionar en un clon donde aún no existe ningún entorno: devuelve dieciséis comprobaciones con el comando de corrección correspondiente al sistema operativo detectado. A continuación solicita **un único dato** — cuál de los tres destinos debe quedar operativo: el cuaderno de entrenamiento, las aplicaciones locales o el despliegue en Databricks — y solo entonces instala, en el orden de dependencia: los datos de LFS, el entorno raíz, los seis entornos de las aplicaciones y la interfaz de línea de comandos de Databricks. **Los componentes ya presentes no se modifican.**
4. **Verificar que los tableros y el simulador operan**. Un diagnóstico satisfactorio no equivale a haber iniciado un tablero. La comprobación se realiza con `/app-local-criticidadCHEC`, o mediante doble clic sobre la aplicación del menú: este abre en el puerto 8800 y desde él se inician los cuatro tableros estáticos (8801 a 8804) y el simulador (8866). **El simulador constituye la verificación determinante**, por ser el único componente que requiere un intérprete de Python en ejecución: marcar un vano y ejecutar una simulación confirma simultáneamente que el entorno está completo, que el modelo carga y que el navegador mantiene la comunicación con el núcleo de cómputo.
5. **Instalar la interfaz de línea de comandos de Databricks**, únicamente para el despliegue. Al seleccionar ese destino, `/instalar-local` la instala y verifica su compatibilidad: no se limita a comprobar su presencia, sino que valida que la creación de aplicaciones acepte el parámetro de tamaño de cómputo, dado que una versión antigua completa las etapas iniciales y falla durante el despliegue. La sesión la abre el usuario con `databricks auth login --host <URL>`, por requerir el navegador. El comando de instalación **no establece comunicación con Databricks**: deja la interfaz instalada y la sesión activa.
6. **Desplegar con `/subir-a-databricks`**. Es el único comando que se comunica con el espacio de trabajo. Solicita un solo dato — su URL, del tipo `https://adb-xxxxxxxxxxxxx.0.azuredatabricks.net` — y la solicita en cada ejecución. De esa URL deriva el perfil y confirma la identidad con una llamada real antes de transferir datos. Si el token está vencido, la ejecución se detiene y solicita una nueva apertura de sesión. Las ocho etapas que recorre después se detallan en la Sección 10.

11.2. Requisitos de máquina, medidos

Las cifras siguientes están medidas sobre el repositorio y no estimadas. El equipo de referencia es un Apple M4 Max de 14 núcleos con 36 GB de memoria y disco de estado sólido. Lo que no cambia entre máquinas es el orden de magnitud y qué parte resulta costosa.

Tabla 29: Requisitos mínimos por componente

Uso previsto	Memoria	Núcleos	Disco libre	Red
Solo los cuatro tableros estáticos	4 GB	2	3,5 GB	Instalación y LFS
Además el simulador	8 GB	4	6 GB	Instalación y LFS
Además reentrenar el modelo	8 GB	2	+3 GB	LFS, una vez
Además generar informes con agentes	8 GB	4	+1 GB	Permanente
Todo, con holgura	16 GB	4–8	20 GB	Permanente

Ninguna parte del proyecto requiere GPU, según la medición reportada en la Tabla 19. La resolución mínima de pantalla para los tableros es de 1.280×800 ; por debajo siguen siendo utilizables y no desbordan, pero las figuras se dibujan por debajo de su tamaño de diseño.

Python 3.11 constituye un piso real y no una preferencia: las versiones requeridas de las bibliotecas numéricas no publican paquetes binarios por debajo de esa versión. El rango se ha ejercitado en ambos extremos — el entorno raíz en 3.11.15 y los seis de las aplicaciones en 3.14.6 —. Ninguna dependencia exige compilador local, dado que existen paquetes binarios precompilados para macOS y Windows de las bibliotecas de mayor tamaño.

11.3. Las cuatro listas de dependencias

Las dependencias se distribuyen en cuatro listas separadas. La separación es deliberada: el menú mantiene un entorno mínimo de aproximadamente 15 MB y no carga las dependencias de los cinco tableros, lo que evita, por ejemplo, requerir la biblioteca de aprendizaje profundo únicamente para iniciar el menú principal.

Tabla 30: Listas de dependencias por componente

Componente	Lista	Dependencias principales
Los cuatro tableros estáticos (8801 a 8804)	<code>aplicaciones/<app>/requirements.txt</code> , una por tablero	<code>pandas</code> , <code>numpy</code> , <code>scipy</code> , <code>scikit-learn</code> , <code>plotly</code> , <code>pyarrow</code> , <code>geopandas</code> , <code>shapely</code> , <code>ipython</code>
El simulador (8866)	<code>aplicaciones/06_simulador/requirements.txt</code>	Las anteriores y, adicionalmente, <code>torch</code> , <code>voila</code> , <code>jupyter-server</code> , <code>ipykernel</code> , <code>ipywidgets</code> , <code>anywidget</code> , <code>joblib</code> , <code>cloudpickle</code> , <code>openpyxl</code> , <code>pyogrio</code> , <code>matplotlib</code>
El cuaderno de entrenamiento	<code>requirements.txt</code> de la raíz	<code>torch</code> , <code>networkx</code> , <code>jupyter</code> , <code>ipykernel</code> , <code>papermill</code> , <code>nbformat</code> , <code>pyproj</code> , <code>folium</code> , además de las dependencias numéricas y geoespaciales
Los informes con agentes	<code>requirements.txt</code> de la raíz	Las del cuaderno y, adicionalmente, <code>pdfplumber</code> , <code>jsonschema</code> , <code>xlsxwriter</code> , <code>graphifyy</code> , <code>openmeteo-requests</code> , <code>requests-cache</code> , <code>retry-requests</code>

La generación de informes depende principalmente de la disponibilidad del servicio de modelo de lenguaje y de la conectividad de red, más que de recursos locales de cómputo. Es el único componente que requiere un servicio externo no incluido en la instalación local. El resto puede ejecutarse sin conexión una vez completada la instalación y disponibles los insumos.

11.4. Comandos de instalación y mantenimiento

Tabla 31: Comandos que cubren el ciclo del entorno

Comando	Función
<code>/instalar-local</code>	Verifica el entorno y prepara los componentes requeridos para las tres operaciones principales: ejecutar el cuaderno de entrenamiento, iniciar el menú con sus cinco tableros y desplegar en Databricks. Diagnostica primero — Python, <code>git-lfs</code> , insumos, entornos, puertos, salida al repositorio de paquetes y la interfaz de Databricks — e instala únicamente los componentes faltantes. Distingue macOS de Windows en cada paso.
<code>/app-local-criticidadCHEC</code>	Inicia el menú desde el cual se lanzan, supervisan y detienen los cinco tableros. Crea el entorno de cada aplicación la primera vez que se requiere.
<code>/actualizar</code>	Detecta cambios en los insumos que afectan al modelo — la base de eventos, el diccionario de variables o el grafo declarado — y reconstruye, en el orden correspondiente, los artefactos afectados: el reentrenamiento, la geometría y los paquetes de las aplicaciones. Si no se detectan cambios, no se ejecuta ninguna acción.

11.5. Portabilidad entre entornos de agente

La instalación no está atada a un proveedor de modelo ni a un editor. Los comandos son contratos en texto — persona, invariantes, secuencia de ejecución y forma de la salida —, de modo que pueden ejecutarse con distintos modelos de lenguaje según la cuota y la política de datos de la organización.

Tabla 32: Ubicación de los comandos y roles en los tres entornos de agente

Entorno	Comandos	Roles y reglas
Claude Code <i>fuentes del contrato</i>	<code>.claude/skills/*/SKILL.md</code> <code>.claude/commands/*.md</code>	y <code>.claude/agents/*.md</code>
OpenCode <i>espejo generado</i>	<code>.opencode/command/*.md</code>	<code>.opencode/agent/*.md</code> , AGENTS.md
VS Code Copilot <i>espejo generado</i>	<code>.github/prompts/*.prompt.md</code>	<code>.github/agents/*.agent.md</code> , AGENTS.md

La fuente única es el directorio de Claude Code; los espejos se generan mediante un guion del repositorio, de forma que un comando renombrado o retirado se propaga en una sola ejecución. La invocación es idéntica en los tres entornos.

Tabla 33: Los quince comandos publicados

Comando	Función
/actualizar	Reconstruye lo afectado cuando cambian los insumos.
/app-local-criticidadCHEC	Abre el menú y sus cinco tableros.
/clima	Enriquece los datos con Open-Meteo.
/expert-alignment	Rol de agente: cotejo contra la discusión experta.
/historical	Rol de agente: diagnóstico descriptivo de la serie.
/inference	Rol de agente: lectura del modelo predictivo.
/informe-gerencial	Síntesis por banda, con barras y grafo radial.
/instalar-local	Diagnostica la máquina e instala lo que falta.
/limpiar-corridas	Mantenimiento de artefactos de corrida, con confirmación explícita.
/pdf-discussion-extraction	Rol de agente: tabla de discusiones desde informes en PDF.
/redaccion-es	Revisión ortográfica y de redacción de la prosa generada.
/report	Informe HTML de un circuito.
/reporte-lote	Encadena /report sobre una banda de riesgo.
/subir-a-databricks	Despliegue a Databricks, en ocho etapas.
/vault-circuito	Proyecta el circuito a la bóveda y encadena la construcción del grafo.

11.6. Primer recorrido de uso

Completada la instalación, la secuencia siguiente recorre el sistema de principio a fin y remite a la sección que describe cada pieza.

1. **Abrir el menú** con `/app-local-criticidadCHEC`. Desde él se inician y se detienen las cinco aplicaciones (Tabla 5).
2. **Situar el problema.** El tablero de agrupamiento indica en qué vanos se concentra la criticidad, y el ranking de circuitos, en qué circuitos (Sección 6.5). A partir de esta clasificación se determina la banda de riesgo sobre la que conviene trabajar.
3. **Situarlo en el tiempo.** Los tableros de trayectorias muestran si el circuito elegido mejora o se deteriora y en qué ventana, primero a nivel de circuito y después a nivel de vano.
4. **Revisar la condición ambiental.** El tablero climático enfrenta el UITI diario contra la variable meteorológica de interés y sus doce horas previas. Si se requieren datos nuevos, el comando `/clima` los incorpora (Tabla 6).
5. **Simular la intervención.** En el simulador se elige el circuito y la ventana, se marcan los vanos, se ejecuta el diagnóstico, se aplican o se ajustan los valores propuestos, se asignan las actividades de contrato y se ejecuta la simulación (Sección 8).
6. **Archivar el resultado.** La acción de guardar deja el informe HTML y su registro en la carpeta de simulaciones; la de cargar repone una ejecución anterior y vuelve a evaluar el modelo (Sección 8.4).
7. **Documentar.** `/report` produce el informe técnico del circuito; `/informe-gerencial`, la síntesis de la banda (Sección 9).
8. **Publicar en la nube**, si corresponde, con `/subir-a-databricks` (Sección 10).

Cuando cambien los insumos — la base de eventos, el diccionario de variables o el grafo declarado —, `/actualizar` reconstruye en orden los artefactos afectados; las aplicaciones, por su parte, detectan el cambio por sí solas y se reconstruyen al abrirse.

12. Grafo de conocimiento del proyecto

El grafo de conocimiento del repositorio se construye sobre el código y la documentación mediante una herramienta de indexación versionada junto al proyecto. El grafo completo comprende 9.976 nodos y 17.025 aristas, cantidad que dificulta su inspección simultánea, por lo que se publica su vista agregada: 754 comunidades y las 877 relaciones entre ellas. Cada punto corresponde a un grupo de símbolos que el algoritmo identificó como unidos, y el número asociado indica cuántos símbolos contiene.

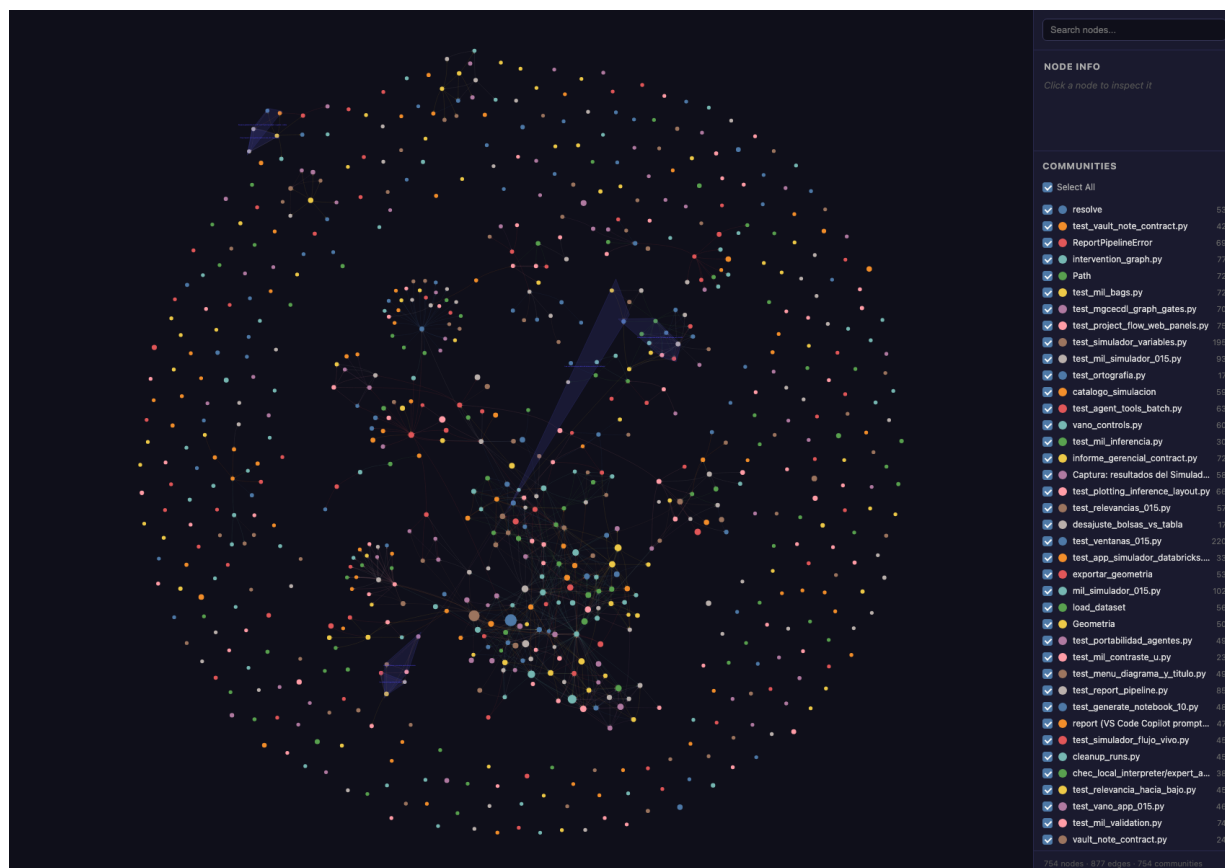


Figura 19: Vista agregada del grafo de conocimiento del proyecto: 754 comunidades y las 877 relaciones entre ellas. La visualización es interactiva y permite buscar símbolos por nombre, seleccionar nodos para consultar su contenido y filtrar comunidades mediante los controles laterales.

Tres precisiones acompañan esta representación:

- **Agregación por comunidades.** Representar simultáneamente alrededor de diez mil nodos reduce de forma significativa la legibilidad en el navegador. La vista agregada conserva la estructura de relaciones y la hace navegable.
- **Etiquetado automático.** Las comunidades proceden de un algoritmo y no incluyen nombre. El nombre mostrado se asigna posteriormente mediante un proceso automático de etiquetado basado en su contenido.
- **Se versiona con el código.** El grafo reside en el repositorio y se versiona con Git, de modo que cada reconstrucción queda registrada como un cambio revisable y no como un archivo transitorio.

Su aplicación práctica es identificar dependencias, consumidores de código y componentes

potencialmente afectados por una modificación. Ese análisis permitió también identificar y retirar componentes sin consumidores activos, entre ellos los reportados en la Tabla 4.

13. Pruebas y validación

La validación se organiza por tipos de evidencia. El repositorio incorpora una suite de **3.278 pruebas automatizadas** repartidas en 182 archivos — 3.226 se ejecutan y 52 se omiten por depender de un entorno que no siempre está disponible —, que cubren la carga de datos, la construcción de contexto, los contratos de salida de los agentes, la validación de esas salidas, la alineación experta, la inferencia del modelo, la disposición de las gráficas, las ventanas y la geometría de criticidad, el simulador por vano, el archivo de simulaciones, el catálogo de costos, el empaquetado de las aplicaciones y el contrato del despliegue a Databricks.

Se contemplan además evaluaciones fuera de línea con contextos sintéticos, que verifican que las instrucciones y las respuestas respeten los contratos, la estructura JSON, las variables permitidas, el manejo de datos faltantes y la separación entre explicación histórica, inferencia predictiva y comparación experta.

El protocolo de validación del modelo por bolsas y su resultado se detallan en la Sección 7.7: veredicto no alcanzado, con la brecha frente a la línea base por debajo del ruido de entrenamiento medido.

La publicación web cuenta con flujo de construcción propio y despliegue automático, lo que permite someter resultados a revisión técnica sin depender de una aplicación de servidor activa. Los tableros que requieren núcleo de cómputo vivo se publican por separado como aplicaciones Databricks.

14. Estimación de costo operativo en Azure Databricks

Esta sección complementa el alcance técnico con un orden de magnitud del costo de operar el flujo en la nube. Las tarifas de Azure, Databricks y los proveedores de modelo de lenguaje se incorporaron el 30 de junio de 2026 y deben verificarse contra la tarifa vigente antes de utilizarse para planeación. La estimación asume **una ejecución completa del flujo ya estabilizado**: no incluye iteraciones para afinar la migración, depurar errores, ajustar rutas, instalar dependencias, repetir cuadernos, validar credenciales ni probar configuraciones de clúster.

Se emplean dos siglas propias de la plataforma. DBU (*Databricks Unit*) es la unidad de consumo con la que Databricks factura el procesamiento, adicional al costo de las máquinas virtuales. FMS (*Foundation Model Serving*) es el servicio administrado de Databricks para consultar modelos fundacionales sin desplegarlos por cuenta propia.

Tabla 34: Comparación de escenarios de operación, por ejecución completa del flujo

Escenario	Configuración de cómputo	Modelo de lenguaje	Costo aprox.
Ajuste interactivo	Clúster interactivo con GPU T4 para todo el flujo	Interfaz externa o FMS	USD 5–8
Ajuste interactivo	Igual, con 30 a 60 min de inactividad	Cualquiera	USD 6–9
Producción	Trabajos programados, con CPU y GPU separadas	Interfaz externa o FMS	USD 4–6
Producción degradada	Clúster interactivo con inactividad prolongada o terminación automática inadecuada	Cualquiera	USD 8–15+
Modelo local 32B+	GPU A100 de 80 GB, por hora	Sin servicio externo	USD 4,5–8/h
Modelo local 32B+	La misma, en operación permanente	Sin servicio externo	USD 3.300–5.800/mes

Tres conclusiones se derivan de la comparación. El costo del modelo de lenguaje por interfaz externa o por FMS resulta bajo frente al costo de GPU y DBU — por debajo de USD 0,30 por ciclo base —, de modo que la elección entre proveedores debe tomarse por gobierno, trazabilidad, privacidad y política empresarial, y no por costo. Separar CPU y GPU mediante trabajos programados reduce el costo de la ejecución completa. Y los modelos locales de 32.000 millones de parámetros o superiores deben tratarse como escenario independiente, porque requieren aceleradores dedicados y agregan un costo por hora que no se amortiza salvo que exista un requisito de privacidad o de control del modelo que lo justifique.

Tabla 35: Referencia de tiempos de la cadena secuencial con GPU

Etapa	Tiempo
Enriquecimiento climático	36,5 min
Búsqueda de hiperparámetros (20 ensayos)	2 h
Entrenamiento (100 épocas)	10 min
Evaluación de desempeño	1 min
Análisis por circuito	5 min
Replicación documental	2 h 30 min
Total aproximado	5,38 h

Alcance de esta referencia. Los tiempos de la Tabla 35 corresponden a la cadena completa de nueve cuadernos vigente al momento de la estimación, cuyo componente dominante era la replicación documental. El flujo actual es sustancialmente más ligero: el entrenamiento del modelo por bolsas se resuelve en unos diez minutos y **no requiere GPU** (Tabla 19), y las dos aplicaciones desplegadas no ejecutan entrenamiento alguno. La estimación se conserva como cota superior de planeación y no como el costo del despliegue descrito en la Sección 10.

15. Entregables y enlaces

Tabla 36: Entregables remotos y evidencias disponibles

Entregable	Alcance	Enlace
Repositorio técnico	Modelo por bolsas, simulador por vano, tableros, agentes, costos, despliegue y publicación web. Incluye la suite de 3.278 pruebas automatizadas.	https://github.com/amalvarezme/chec-local-uiti-vano-interpreter
Sitio público técnico	Publicación con las visualizaciones, la arquitectura, el modelo, los agentes y los resultados interactivos.	https://amalvarezme.github.io/chec-local-uiti-vano-interpreter/
Sitio base del proyecto	Página institucional del proyecto CHEC-UNAL.	https://amalvarezme.github.io/CHECUNAL_2026/
Repositorio del tablero	Código del tablero Dash, servicios y ruta de paridad reportada previamente.	https://github.com/amalvarezme/dashboard_chec

16. Conclusiones

El desarrollo consolidó el paso del diagnóstico a la simulación. El sistema ya no se limita a explicar el comportamiento histórico de `UITI_VANO`: responde, para un vano y una ventana concretos, qué variable modificar y hasta qué valor para que descienda de grupo de criticidad, entrega esa respuesta en un tablero interactivo con su costo cotizado sobre el catálogo del contrato, y permite archivar y recuperar cada ejecución. La unidad de análisis quedó alineada con la unidad de decisión, que es el activo sobre el cual se emite una orden de trabajo.

La incorporación del grafo de restricciones físicas como término de la función de costo, y no como validación posterior, mantiene la explicación dentro de las relaciones que el personal técnico reconoce. **La medición asociada indica que, con los pesos actuales, la mayor parte del efecto del grafo llega por la vía de la propagación — que es estructural y actúa siempre — y no por la del término de penalización, cuya contribución al gradiente es pequeña. Cuantificar por separado el aporte de cada una de las dos vías mediante una ablación explícita es el siguiente refinamiento previsto.**

Desempeño del modelo: la lectura correcta. El modelo M-GCECDL alcanza un macro-F1 de 0,8710 sobre 62.114 bolsas evaluadas fuera de pliegue, con acierto del 85,4 %. **Es un resultado competitivo frente a las líneas base de referencia** para datos tabulares. Contra un bosque aleatorio ajustado sobre las mismas variables, la diferencia es de 1,02 puntos de macro-F1, muy por debajo de los siete puntos de dispersión que separan dos corridas idénticas del propio entrenamiento: en términos de acierto, **ambos modelos son equivalentes**. Contra las líneas base que indican si hay señal aprendible — persistencia de la clase anterior y clase mayoritaria — la ventaja supera los cuarenta puntos.

Con desempeño equivalente al de esas líneas base, y frente a arquitecturas tabulares profundas del tipo TabNet, la propuesta se distingue por tres razones que sí son diferencias de fondo:

- **Está mucho más ajustada a las restricciones físicas y temporales del dominio.** La información solo puede propagarse por las 156 relaciones que la ingeniería declaró en

el grafo, y la unidad de análisis es la bolsa vano $\times$ ventana con agregación invariante a la cantidad de registros. Una explicación imposible en la red eléctrica es también imposible dentro del modelo, y el número de eventos no puede colarse como predictor por una vía indirecta.

- **Es notablemente más eficiente.** 150.926 parámetros, un artefacto de 691 KB que se transporta completo, y un reentrenamiento íntegro en unos diez minutos sobre una CPU de dos núcleos, sin GPU en ninguna etapa.
- **Es mucho más interpretable.** Entrega, para cada vano y cada ventana, qué variables pesaron y cuánto, por qué esa relación estaba permitida, y qué cambio concreto haría descender al vano de grupo de criticidad, con su costo tomado del catálogo del contrato. Ninguna de las líneas base produce esa cadena.

Es decir: **resultados comparables, con un modelo sustancialmente más pequeño, más rápido, más restringido por la física del problema y capaz de sostener una recomendación operativa explicable.** Esa combinación es la que hace utilizable el resultado, porque una priorización de intervenciones necesita justificarse ante quien la ejecuta.

El perfil de error refuerza esa aptitud operativa: la clase Alto — la que dispara la intervención — es la mejor predicha, con F1 de 0,958, y los errores se concentran en clases contiguas. La prioridad técnica inmediata es acotada y de ingeniería: fijar el determinismo del entrenamiento, para que las comparaciones finas entre variantes se puedan sostener con confianza.

Tres decisiones sostienen la trazabilidad del conjunto y conviene mantenerlas: la geometría de criticidad se congela y se verifica por firma, de modo que el mapa observado y el simulado son comparables por construcción; el efecto simulado y el costo cotizado permanecen desacoplados, porque unirlos sin respaldo produciría una cifra con apariencia de estimación de beneficio; y el razonamiento en lenguaje natural queda fuera del código Python, sujeto a contratos de salida que se validan antes de publicar.

La combinación de estos elementos con la copia reproducible del dominio en Databricks y con la instalación asistida constituye una base verificable para la priorización de intervenciones y la generación de reportes de evidencia.

Dir. Proyecto UNAL
Prof. Andrés Marino Álvarez Meza
Departamento de Ingeniería Eléctrica, Electrónica y Computación
Universidad Nacional de Colombia Sede Manizales